

Disclosure Pilot AI

by

Ronald Cook

www.linkedin.com/in/ronald-cook-programmer

August 2026

Disclosure Pilot AI

Introduction

The primary objective of this project is to demonstrate the feasibility of using Large Language Models (LLMs) to build a fully functional, end-to-end software application. To showcase this capability, a sample program was developed for accountants to generate SEC Form 10-Q and Form 10-K financial reports. The technical architecture of this application consists of a React frontend, a FastAPI backend, a PostgreSQL database, and Arelle Viewer.

Document Structure

This report is organized into distinct sections:

- **Section 1: What are 10-K and 10-Q Filings?** – Explains the difference between a company's annual report, Form 10-K, and Form 10-Q.
- **Section 2: React Frontend** – Demonstrates the software's functionality and user interface through detailed screenshots.
- **Section 3: Arelle Viewer** – Illustrates the operation of the Arelle validation tool, which highlights iXBRL tags within the HTML filing.
- **Section 4: Architecture & Scope** – Explains the framework and limited scope of this program.
- **Section 5: Database Schema** – Lists the structure of the PostgreSQL database tables, which contain the financial accounting data.
- **Section 6: Accounting Fundamentals** – Reviews the foundational accounting principles and background relevant to the application.
- **Section 7: LLM Analysis** – Evaluates the current state of Large Language Models, separating practical capabilities from industry hype.
- **Section 8: AI Agent Tutorial** – Describes the AI agent employed to perform two particular functions in this application, Management's Discussion and Analysis (MD&A) and Notes to Financial Statements.
- **Section 9: Data Generation Prompting** – Outlines the methodologies used to prompt Meta AI to generate realistic, sample financial data. (Not required in real business environment with existing database.)

- **Section 10: Powershell Script Prompting** – Describes how to prompt Claude to generate application Powershell scripts. Also, shows prompt for Gemini to create script explanations.
- **Section 11: Code Development Prompting** – Explains the prompt engineering strategies used with Claude to generate the core codebase for the application.
- **Section 12: iXBRL Prompting** – Specifies the iterative prompt strategies used with Claude to generate an iXBRL transformer for the application.
- **Section 13: Testing** – Outlines how the application code and financial dataset were tested.
- **Section 14: Tools** – Provides an overview of each software tool.
- **Section 15: Discussion** – Examines significant issues associated with the application.
- **Conclusion: Key Takeaways for LLM-Assisted Development** – Highlights importance of understanding LLM prompting dynamics and iterative strategies.

Target Audience

This document is designed for individuals navigating the integration of AI into software engineering, specifically:

- **Aspiring Developers** who leverage AI assistants for basic tasks, but feel hesitant to architect end-to-end systems with complex LLM models.
- **Industry Skeptics** doubting the capability of AI to generate high-quality, production-grade code.
- **STEM Students** who rely on AI to complete assignments but may struggle to build real-world applications because they have not learned proper software design principles.
- **Executives & CEOs** slashing development teams to cash in on AI gains are ignoring history: massive productivity breakthroughs consistently trigger higher demand, expand total employment, and shorten the workweek.

Note: Google Gemini and Meta AI were used to assist in drafting portions of this document.

Section 1 - What are 10-K and 10-Q Filings?

It's incredibly common to hear people use "10-K" and "Annual Report" interchangeably, but they actually serve different audiences and have distinct purposes.

Think of it this way: the **Form 10-K is the raw, unvarnished data** required by the government, while the **Annual Report is the company's glossy storytelling piece** meant for the public.

Here is exactly how they differ across the key areas:

Feature	Form 10-K	Glossy Annual Report
Primary Audience	Regulators (SEC), institutional investors, and financial analysts.	Shareholders, potential customers, employees, and the general media.
Format & Tone	Strictly text and numbers. Dense, legalistic, and completely devoid of marketing fluff or color.	Highly designed. Features photographs, corporate branding, infographics, and a letter from the CEO.
Key Focus	Detailed financial disclosures, legal risks, internal controls, and granular market data.	Corporate vision, strategic goals, highlights of the past year, and future market opportunities.
Filing Requirement	Mandatory by law to be filed with the SEC within 60 to 90 days of the fiscal year-end.	Not technically required by the SEC, though usually distributed ahead of the annual shareholder meeting.

Why the confusion?

The line gets blurry because of something called the **"Wrap-Around" method**.

To save money and time on printing, many companies will write a glossy, beautifully designed opening section (with the letter from the CEO and a summary of performance) and literally "wrap" it around their actual, legal Form 10-K filing.

So, if you download a company's Annual Report and it suddenly shifts from a colorful magazine layout to hundreds of pages of black-and-white financial tables, you are looking at a combined document.

Which one should you read?

- Go to the **Annual Report** if you want to understand management's high-level philosophy, how they view their competitors, and what new products they are excited about launching.
- Go to the **Form 10-K** if you want to dig into the cold, hard numbers, see exactly how much debt they owe, or read the "Risk Factors" section where they legally must disclose everything that could potentially bankrupt the company.

Difference between Form 10-K and Form 10-Q

The core difference comes down to **scope** and **frequency**. Think of a **Form 10-K** as a company's comprehensive **yearly** biography, while a **Form 10-Q** is a quick, **quarterly** progress report.

Both are mandatory filings required by the U.S. Securities and Exchange Commission (SEC) for publicly traded companies, but they serve different purposes for investors.

Key Differences at a Glance

Feature	Form 10-K	Form 10-Q
Frequency	Once a year (covers the full fiscal year)	Three times a year (covers Q1, Q2, and Q3)
Audit Status	Audited by an independent CPA firm	Unaudited (reviewed, but not fully audited)
Size & Detail	Massive and deeply detailed	Shorter, focusing mainly on recent changes
Key Inclusions	Corporate history, executive pay, full market risk analysis, audited financial statements	Basic financial statements, updates on legal proceedings, minor risk updates

Form 10-K: The Deep Dive

Because the 10-K is published at the end of the fiscal year, it provides a panoramic view of the company's financial health. It includes a detailed narrative of the business operations, market share, risk factors, and legal struggles.

Most importantly, the financial statements in a Form 10-K **must be audited** by an outside accounting firm. This audit gives investors a high level of confidence that the numbers haven't been manipulated.

Form 10-Q: The Pulse Check

The Form 10-Q is designed to give investors a continuous look at how the company is doing throughout the year. It focuses strictly on the financial results of the latest quarter and compares them to the same quarter from the previous year.

Because companies have to put these together quickly every three months, the SEC **does not require them to be audited**. They are **only reviewed** by accountants, meaning they are a bit more raw and subject to adjustments later in the year.

Note: There is no Form 10-Q for the fourth quarter. The data for Q4 is rolled directly into the massive, final Form 10-K report.

Note: This application is focused on creating the Form 10-K and Form 10-Q reports, not the glossy annual report.

Form 10-K Table of Contents

The official SEC Form 10-K is divided into **four distinct parts** spanning **15 core items** (plus a few specialized sub-items like Cybersecurity and Mine Safety). The standardized layout used by publicly traded companies in the United States is outlined below:

Part I

- **Item 1.** Business (*Company overview, products/services, markets, and regulations*)
- **Item 1A.** Risk Factors (*Potential threats, vulnerabilities, and market challenges*)
- **Item 1B.** Unresolved Staff Comments (*Outstanding questions or issues raised by SEC staff*)

- **Item 1C.** Cybersecurity (*Risk management, strategy, governance, and incident disclosures*)
- **Item 2.** Properties (*Physical assets, plants, factories, and real estate holdings*)
- **Item 3.** Legal Proceedings (*Significant ongoing or pending lawsuits*)
- **Item 4.** Mine Safety Disclosures (*Applicable only to mining operators*)

Part II

- **Item 5.** Market for Registrant's Common Equity, Related Stockholder Matters and Issuer Purchases of Equity Securities (*Stock performance, dividends, and share buybacks*)
- **Item 6.** [Reserved] (*Formerly Selected Financial Data; deleted by the SEC to eliminate redundancy*)
- **Item 7.** Management's Discussion and Analysis of Financial Condition and Results of Operations (MD&A) (*Management's narrative explanation of the financial results, trends, and liquidity*)
- **Item 7A.** Quantitative and Qualitative Disclosures About Market Risk (*Exposure to interest rates, foreign currency exchange, or commodity price fluctuations*)
- **Item 8.** Financial Statements and Supplementary Data (*Audited balance sheets, income statements, cash flows, and footnote disclosures*)
- **Item 9.** Changes in and Disagreements with Accountants on Accounting and Financial Disclosure
- **Item 9A.** Controls and Procedures (*Evaluation of internal controls over financial reporting*)
- **Item 9B.** Other Information (*Disclosures that would typically require a Form 8-K but occurred during the fourth quarter*)
- **Item 9C.** Disclosure Regarding Foreign Jurisdictions that Prevent Inspections

Part III

Note: Companies often "incorporate by reference" the information in Part III from their definitive proxy statement rather than reprinting it here, provided the proxy is filed within 120 days of the fiscal year-end.

- **Item 10.** Directors, Executive Officers and Corporate Governance

- **Item 11.** Executive Compensation
- **Item 12.** Security Ownership of Certain Beneficial Owners and Management and Related Stockholder Matters
- **Item 13.** Certain Relationships and Related Transactions, and Director Independence
- **Item 14.** Principal Accountant Fees and Services (*Fees paid to the independent auditing firm*)

Part IV

- **Item 15.** Exhibit and Financial Statement Schedules (*The full list of legal documents, material contracts, and bylaws filed alongside the report*)
- **Item 16.** Form 10-K Summary (*An optional section allowing companies to provide a brief summary index*)

Reading Strategy Tip: If you are analyzing a company's financial health, experienced investors usually skip straight to Item 7 (**MD&A**) to see how management explains the numbers in their own words, followed by Item 8's **footnotes** (also called **Notes**) to check for hidden liabilities or shifting accounting policies.

Notes: GAAP disclosures

While this application focuses on only 8 footnotes for simplicity, most corporate filings list 15 to 20 footnotes. Even though individual companies may alter the exact wording slightly based on their industry or specific operations, U.S. GAAP and SEC regulations mandate highly structured disclosures.

If you look across any major SEC Form 10-K, these are the **15 most common, standardized names** for the Notes (footnotes) to the Consolidated Financial Statements:

1. Summary of Significant Accounting Policies

What it covers: The foundational "rules of the road" for how the company prepares its books. It details the basis of consolidation, the use of estimates, and brief summaries of depreciation and foreign currency translation methods.

2. Revenue (or Revenue from Contracts with Customers)

What it covers: Details the implementation of Accounting Standards Codification (ASC) 606. It explains *when* and *how* the company transfers control of goods or services, breaks

down revenue by product lines or geographic regions, and explains contract assets/liabilities (like deferred revenue).

3. Debt (or Financing Arrangements)

What it covers: Details both short-term and long-term borrowing. It lists maturity schedules, interest rates, collateral requirements, and debt covenants (which can trigger defaults if breached).

4. Income Taxes

What it covers: Reconciles the difference between the statutory federal tax rate and the company's effective tax rate. It also discloses **deferred tax assets and liabilities** (such as Net Operating Losses) and reserves for uncertain tax positions.

5. Property, Plant, and Equipment, Net

What it covers: Breaks down physical assets (land, buildings, machinery). It discloses the cost, accumulated depreciation, useful lives utilized, and any asset impairment charges taken during the year.

6. Leases

What it covers: Governed by ASC 842. It breaks down operating and finance lease liabilities, right-of-use (ROU) assets, lease terms, discount rates, and future maturity pipelines.

7. Goodwill and Intangible Assets

What it covers: Breaks down non-physical assets (such as patents, trademarks, and customer lists) alongside goodwill from past acquisitions. It details the annual impairment testing process and any write-downs.

8. Commitments and Contingencies

What it covers: Reveals off-balance-sheet liabilities. It is heavily scanned by analysts to locate pending legal proceedings, regulatory fines, environmental liabilities, and purchase commitments.

9. Share-Based Compensation (or Stock-Based Compensation)

What it covers: Outlines how the company compensates employees using equity restricted stock units (RSUs) or stock options. It discloses the fair value assumptions (like Black-Scholes variables) and total compensation expense.

10. Segment Reporting (or Segment Information)

What it covers: Required if a company manages its business through multiple operational divisions. It breaks down revenue, operating income, and assets by operating segments as reviewed by the Chief Operating Decision Maker (CODM).

11. Fair Value Measurements

What it covers: Breaks down how the company values its financial instruments (investments, derivatives, etc.) using the three-level hierarchy (Level 1: observable market prices, Level 2: indirect inputs, Level 3: unobservable/internally modeled inputs).

12. Pension and Other Postretirement Benefits

What it covers: Discloses the funded status of defined-benefit pension plans or other post-employment benefits (OPEB). It breaks down the plan assets, projected benefit obligations, and demographic/discount rate assumptions.

13. Acquisitions and Divestitures (or Business Combinations)

What it covers: Explains the financial details of any company purchased or sold during the fiscal year. It includes the purchase price allocation (how much went to cash, inventory, Intellectual Property (IP), and goodwill) and pro forma revenue.

14. Related Party Transactions

What it covers: Discloses transactions with executives, directors, major shareholders, or affiliate companies. It ensures transparency regarding potential conflicts of interest or non-arm's-length terms.

15. Subsequent Events

What it covers: Discloses any material events that occurred *after* the balance sheet date but *before* the 10-K was officially issued. Examples include post-close acquisitions, debt refinancing, or devastating legal rulings.

This application implements only 8 common 10-K Item 8 Notes, listed below:

Basis of Presentation

Summary of Significant Accounting Policies

Segment Information

Income Taxes

Earnings Per Share

Commitments and Contingencies

Short-term and Long-term Debt

Leases

Form 10-K References:

<https://www.sec.gov/files/form10-k.pdf>

<https://www.sec.gov/files/reada10k.pdf#:~:text=Data%E2%80%9D%20requires%20the%20company's%20audited%20financial%20statements.,the%20information%20presented%20in%20the%20financial%20statements.>

<https://asc.fasb.org/Login>

<https://asc.fasb.org/105/showallinonepage>

<https://www.colonialfilings.com/edgar-downloadable-form-templates/#Form10-K>

<https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins/how-read>

Form 10-Q Table of Contents

Unlike the extensive yearly Form 10-K, the **Form 10-Q** is the streamlined **quarterly** financial report required by the SEC. It is much tighter, focusing strictly on updating investors on the company's financial status and material changes over the last 3 months. It consists of just **two main parts** spanning **11 total items**:

Part I – Financial Information

- **Item 1.** Financial Statements (*Unaudited balance sheets, income statements, and cash flows for the quarter*)
- **Item 2.** Management's Discussion and Analysis of Financial Condition and Results of Operations (MD&A) (*Management's narrative explaining the quarterly performance, revenue changes, and liquidity*)

- **Item 3.** Quantitative and Qualitative Disclosures About Market Risk (*Updates on exposure to market factors like interest rates or currency fluctuations*)
- **Item 4.** Controls and Procedures (*The quarterly evaluation of disclosure controls and internal reporting mechanisms*)

Part II – Other Information

- **Item 1.** Legal Proceedings (*Updates on new material lawsuits or significant developments in existing legal battles*)
- **Item 1A.** Risk Factors (*New or significantly modified risks since the last 10-K; smaller reporting companies often omit this unless a major shift occurred*)
- **Item 2.** Unregistered Sales of Equity Securities and Use of Proceeds (*Details on stock buybacks or private sales of shares*)
- **Item 3.** Defaults Upon Senior Securities (*Disclosures if the company fell behind on paying interest/principal on major debts*)
- **Item 4.** Mine Safety Disclosures (*Applicable only to mining companies*)
- **Item 5.** Other Information (*Disclosures that would have triggered a Form 8-K but happened to occur during the quarter, including insider trading arrangement disclosures under Rule 10b5-1*)
- **Item 6.** Exhibits (*The legal attachments, executive certifications like Sarbanes-Oxley, and interactive financial data files*)

The Big Difference: The Form 10-Q financial statements in **Part I, Item 1** are **unaudited**. While an independent CPA firm must **review** the Form 10-Q numbers before filing, they do not undergo the full, rigorous audit required for the Form 10-K.

Form 10-Q References:

<https://www.sec.gov/files/form10-q.pdf>

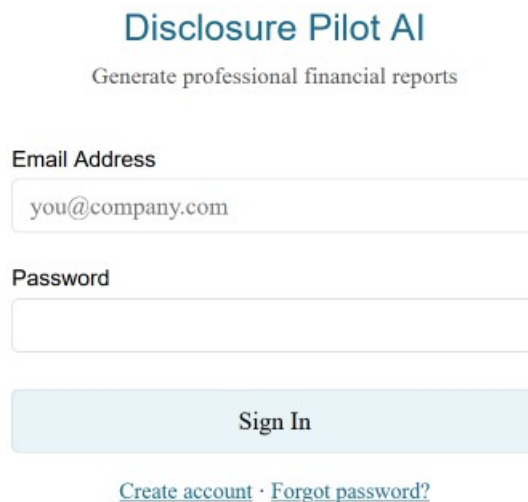
<https://www.quantillum.com/blog/sec-form-10-q>

Section 2 - React Frontend

This section illustrates how the application works, by walking through each screenshot, from login to final document creation.

Running the Application

To setup and run the application see the Start-Here-Setup.docx file, under docs/installation.



The screenshot shows the login interface for 'Disclosure Pilot AI'. At the top, the title 'Disclosure Pilot AI' is displayed in a teal font, with the tagline 'Generate professional financial reports' below it in a smaller, grey font. The login form consists of two input fields: 'Email Address' with the placeholder text 'you@company.com' and 'Password'. Below these fields is a light blue 'Sign In' button. At the bottom of the form, there are two links: 'Create account' and 'Forgot password?', both in a teal color.

Enter default email and password:

email: zx@qaz.com, and password: qwerty

After a successful sign in, the Home Page should open.



The screenshot shows the home page of the application after a successful login. The top navigation bar is light blue and contains the 'Disclosure Pilot AI' logo on the left, and links for 'Home', '10-Q', '10-K', and 'Help' on the right. The '10-Q' and '10-K' links have dropdown arrows. The main content area has a light yellow background. It features a large teal heading 'Welcome to Disclosure Pilot AI', followed by the tagline 'Generate professional quarterly and yearly financial reports. Signed in as' and the user's email 'zx@qaz.com'.

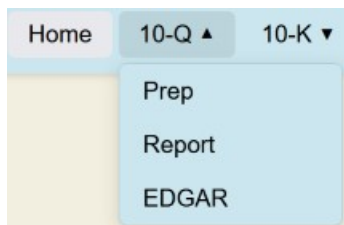
The menu structure is:

Home

10-Q: Prep, Report, EDGAR

10-K: Prep, Report, EDGAR

Help: User Guide, About



Workflow 10-Q 2025 Q2

1. **10-Q Prep page** → Select year, 2025 + quarter, 2 → Load financial statements
2. Review balance sheet, income statement, cash flow, and stockholders equity tables
3. Click **Save Financials** → creates item010-Financial-Statements.docx in sec10q-q2/part1/
4. Click **Generate MD&A** → AI generates narrative via LangGraph + Ollama
5. Edit narrative in the text area
6. Click **Save MD&A** → creates item020-MDA.docx in sec10q-q2/part1/
7. Click **Generate Notes** → AI generates narrative via LangGraph + Ollama
8. Edit narrative in the text area
9. Click **Save Notes** → creates item011-Notes-to-Financial-Statements.docx in sec10q-q2/part1/
10. **10-Q Report page** → Click **Generate Form 10-Q Report** → merges all docx item files into sec-10q-2025-q2.docx
11. **10-Q EDGAR page** → Click **Generate 10-Q EDGAR HTM(iXBRL)** → converts sec-10q-2025-q2.docx to sec-10q-2025-q2.htm

The 10-Q Prep page allows the user to review the 10-Q balance sheets, income statement, cash flow, and stockholders' equity.

Prepare 10-Q Quarterly Report

Select Period

Year

2024



Quarter

Q2 (Apr-Jun)



Load Financial Statements

Balance Sheets

	June 30, 2025	Year ended December 31, 2024
ASSETS		
Cash and Equivalents	\$71,206,456	\$59,475,456
Short-Term Investments	14,249,430	11,155,182
Accounts Receivable	26,849,498	20,540,624
Allowance for Doubtful Accounts	(4,498,960)	(3,417,720)
Inventory	3,780,000	2,940,000
Prepaid Expenses	2,270,001	1,748,600
Property, Plant & Equip	9,423,509	8,971,128
Accumulated Depreciation	(1,829,502)	(1,418,688)
Total Assets	121,450,431	99,994,581
LIABILITIES		
Accounts Payable	\$5,787,375	\$4,571,875
Accrued Liabilities	263,062	207,813
Short-term Debt	223,616	285,746

Statements of Income

	Three Months Ended June 30, 2025	Three Months Ended June 30, 2024	Six Months Ended June 30, 2025	Six Months Ended June 30, 2024
REVENUE				
Hardware Sales	\$14,175,000	\$9,450,000	\$28,350,000	\$18,900,000
Software Sales	11,756,250	7,837,500	22,978,125	15,140,625
Consulting	375,000	375,000	812,000	812,000
Total Revenue	26,306,250	17,662,500	52,140,125	34,852,625
COST OF GOODS SOLD				
Cost of Goods Sold	(5,670,000)	(3,780,000)	(11,340,000)	(7,560,000)
Gross Profit	20,636,250	13,882,500	40,800,125	27,292,625
OPERATING EXPENSES				
Salaries and Wages	3,088,125	2,058,750	6,176,250	4,117,500
Supplies Expense	263,063	182,271	521,401	354,172
Lease Expense	300,000	300,000	600,000	600,000
Bad Debt Expense	540,000	360,000	1,081,240	721,240

Statements of Cash Flows		
	Six Months Ended June 30, 2025	Six Months Ended June 30, 2024
CASH FLOWS FROM OPERATING ACTIVITIES		
Net Income	\$18,799,019	\$12,079,820
Depreciation Expense	410,814	327,925
Bad Debt Expense	1,081,240	721,240
Changes in Accounts Receivable	(6,308,875)	(4,071,375)
Changes in Inventory	(840,000)	(420,000)
Changes in Prepaid Expenses	(521,401)	(354,172)
Changes in Accounts Payable	1,215,500	810,333
Changes in Accrued Liabilities	55,250	36,833
Changes in Unearned Revenue	2,671,875	1,959,375
Net Cash from Operating Activities	16,563,422	11,089,980
CASH FLOWS FROM INVESTING ACTIVITIES		
Purchase of Short-Term Investments	(3,094,248)	(1,647,415)
Purchase of Property, Plant & Equip	(452,381)	(184,161)
Net Cash from Investing Activities	(3,546,629)	(1,831,576)

Statements of Stockholders' Equity					
	Common Stock	Treasury Stock	Retained Earnings	Accumulated Other Comprehensive Income (Loss)	Total Stockholders' Equity
Three Months Ended					
Balance as of March 31, 2025	\$28,000,000	(\$2,490,625)	\$61,840,747	—	\$87,350,122
Net income	—	—	10,468,028	—	10,468,028
Dividends declared	—	—	(54,974)	—	(54,974)
Purchase of treasury stock	—	(54,974)	—	—	(54,974)
Balance as of June 30, 2025	\$28,000,000	(\$2,545,599)	\$72,253,801	—	\$97,708,203
Balance as of March 31, 2024	\$27,000,000	(\$1,398,995)	\$34,903,027	—	\$60,504,032
Net income	—	—	6,717,337	—	6,717,337
Dividends declared	—	—	(43,215)	—	(43,215)
Purchase of treasury stock	—	(43,215)	—	—	(43,215)
Balance as of June 30, 2024	\$27,000,000	(\$1,442,210)	\$41,577,149	—	\$67,134,939

Before generating the Management Discussion and Analysis (MD&A), the MD&A Factors panel allows the user to edit key elements as input to the analysis, depending on the financial results. For example, if net income is positive, then Strategic Initiatives could be “investing in R&D”. On the other hand, if quarterly net income is negative, Strategic Initiatives could be “cost cutting measures”. Risk Factors may apply to both scenarios. Critical Accounting Estimates consider special cases, such as changes in income tax rules.

MD&A Factors

Strategic Initiatives

investing in R&D, expanding into new markets

Risk Factors

increased competition, changes in market conditions

Critical Accounting Estimates

income tax allowances, useful life of assets

Generate MD&A

When the user clicks on Generate MD&A, an AI agent function uses the financial statements and company information to create the MD&A narrative, which the user may then edit. An excerpt is shown below.

Liquidity and Capital Resources

As of June 30, 2025, our total assets stood at \$121.5 million, representing a significant increase from the \$100.0 million reported as of June 30, 2024. This growth in assets is primarily attributed to the robust operating cash flow of \$16.6 million for the quarter ended June 30, 2025, compared to \$11.1 million in the same period last year. The improved operating cash flow reflects enhanced operational efficiency and effective management of working capital.

Our liquidity position remains strong with a current ratio of 5.12x, indicating that we have sufficient short-term assets to cover our immediate liabilities. This ratio is consistent with our strategic focus on maintaining robust liquidity buffers to support ongoing operations and future growth initiatives. Additionally, our total equity stood at \$97.7 million as of June 30, 2025, underscoring the financial health and stability of the company.

Critical Accounting Policies

The Company's financial statements are prepared in accordance with Generally Accepted Accounting Principles (GAAP) and require management to make certain critical accounting estimates, particularly related to income tax allowances. These estimates significantly impact our effective tax rate and overall financial results.

After completing the MD&A edit, the user clicks on Submit MD&A to save the narrative to a specific file that will become part of the 10-Q report.

Submit MD&A

MD&A saved to item020-MDA.docx

The Notes to Financial Statements panel lists the notes input by the user on the file, `note_list_10q-q2.toml`, under `AppIO/user_input`. There is a separate file for each quarter, because financial conditions may change from quarter to quarter.

Notes to be Included

1. Basis of Presentation
2. Segment Information
3. Income Taxes
4. Earnings Per Share
5. Commitments and Contingencies

Generate Notes

When the user clicks on Generate Notes, an AI agent function uses the financial statements and company information to create the Notes narrative, which the user may then edit. An excerpt is shown below.

Income tax expense for the six months ended June 30, 2025 was \$2,321,994, representing an effective tax rate of 11.0% of pre-tax income of \$21,121,013.

Note 4 – Earnings Per Share

TIME FLUX LLC has no outstanding potentially dilutive securities such as stock options, warrants, or convertible instruments that would affect the calculation of diluted earnings per share (EPS) for the period ended June 30, 2025. As a result, basic EPS and diluted EPS are identical.

Basic earnings per share was \$1.8799, calculated by dividing net income of \$18,799,019 by the weighted average number of outstanding common shares of 10,000,000. Diluted earnings per share was also \$1.8799.

Note 5 – Commitments and Contingencies

As of June 30, 2025, Time Flux LLC is not subject to any material legal proceedings other than ordinary routine litigation incidental to its business. The company has no significant non-cancelable purchase commitments or guarantees of third-party obligations outside the ordinary

After completing the Notes edit, the user clicks on Submit Notes to save the narrative to a specific file that will become part of the 10-Q report.

Submit Notes

Notes saved to `item011-Notes-to-Financial-Statements.docx`

The Report 10-Q page allows the user to merge all the separate 10-Q items into the 10-Q report, and save to a file.

Caution: Since several docx files may be prepared externally to this application, the user must ensure that all items are **completed and approved** before creating the final 10-Q report.

Generate Form 10-Q Report

Select Period

Caution: Ensure all Form 10-Q item files have been completed before generating the final report.

Year

Quarter

2025

Q2 (Apr-Jun)

Generate Form 10-Q Report

10-Q report generated: sec-10q-2025-q2.docx

The 10-Q EDGAR page allows the user to convert the 10-Q report into an Edgar HTML file, ready to process with Arelle validator and viewer, illustrated in the next section.

Select Report

Converts a previously generated Form 10-Q report into SEC EDGAR-compliant iXBRL format. Pick which period's report to convert below — the Form 10-Q DOCX report for that period must already exist (see Generate Form 10-Q Report).

☐ sec-10q-2025-q3.docx

☒ sec-10q-2025-q2.docx

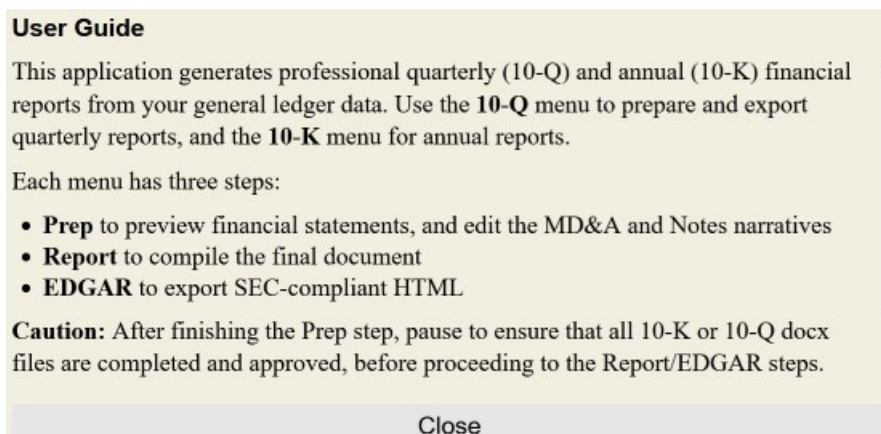
☐ sec-10q-2025-q1.docx

Generate 10-Q EDGAR HTM(iXBRL)

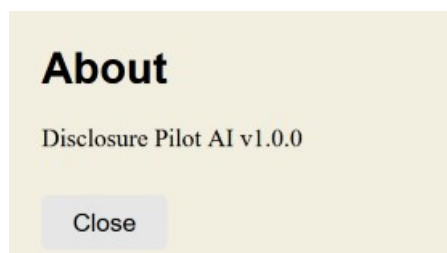
EDGAR HTM generated: sec-10q-2025-q2.html + iXBRL -> sec-10q-2025-q2.htm

The 10-K prep, report, and EDGAR pages match the 10-Q pages in how they work, and are not shown here.

The User Guide dialog briefly describes the application menus.



The About dialog just displays the application version.



React Client references:

<https://thepythoncode.com/article/fullstack-notes-app-with-fastapi-and-reactjs>

<https://www.bairesdev.com/blog/react-spa-single-page-application/>

<https://www.geeksforgeeks.org/python/fastapi-templates/>

Fastapi Server references:

<https://realpython.com/fastapi-python-web-apis/>

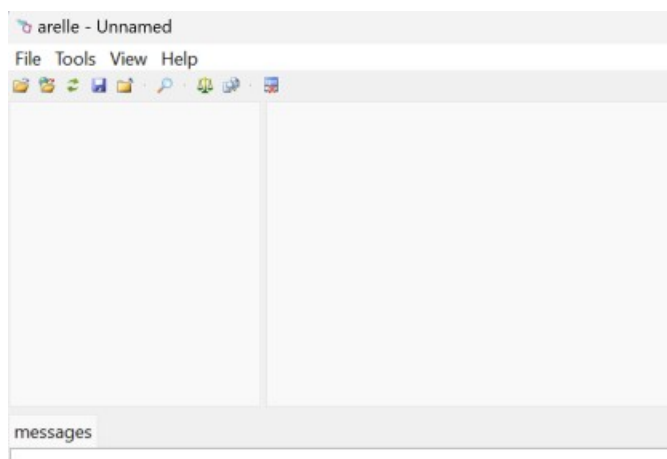
<https://fastapitutorial.com/blog/serving-html-fastapi/>

Section 3 – Arelle Viewer

Arele for Windows is a free, open-source desktop application designed for managing XBRL data. Navigating Arelle's interface can feel a bit overwhelming at first because it maps out highly technical, multi-layered XBRL structures. Powered by its built-in iXBRL Viewer plugin, it allows users to interactively view, validate, and analyze Inline XBRL(iXBRL) files directly within modern web browsers. While the upcoming section explores iXBRL in full detail, this section focuses purely on demonstrating how Arelle extracts and highlights data within HTML filings.

Settings

From the Windows app list select Arelle, and the desktop app opens as illustrated below:



Before looking at our 10-K file, we need to choose some settings.

Under **Tools** → **Validation**:

- choose “Disclosure system checks”.
- Select disclosure system: choose “US SEC (Edgar Filing Manual Pragmatic)”.
- Base taxonomy validation: select “Use disclosure system settings”.
- choose “Unit Type Registry validation”. It prevents errors like using meters for Profit, or using USD for Area.
- choose “OIM validate xBRL-XML documents”. Calculations 1.1 requires OIM - Open Information Model, so conversion between XML <-> JSON <-> CSV is lossless.

- do not choose “Validate all files as Report Packages”. For just validating a single htm file directly, leave it off. A report package is a zip file that contains one or more XBRL, Inline XBRL, xBRL-CSV, xBRL-JSON. If the zip contains META-INF/reportPackage.json it's a Report Package.
- do not choose “Validate all files as Taxonomy Packages”. A Taxonomy Package holds a *taxonomy* itself - the schemas + linkbases that define the reporting rules (.xsd, -lab.xml, -pre.xml, etc). If the ZIP contains META-INF/taxonomyPackage.xml it's a Taxonomy Package. If just validating a normal filing (report.htm), leave it off.
- Warn on duplicate facts: select “**Inconsistent**”. In the messages panel, Arelle uses a single format to list both inconsistent and consistent warnings. Because consistent warnings are not errors, they should be ignored. Only inconsistent warnings should be flagged and fixed.

Under Tools → Calc linkbase: select “Calc 1.1 round to nearest mode”.

Under Tools → iXBRL Viewer → Settings: select “Launch on Load”, “Copy Script”, and “Highlight facts on startup”.

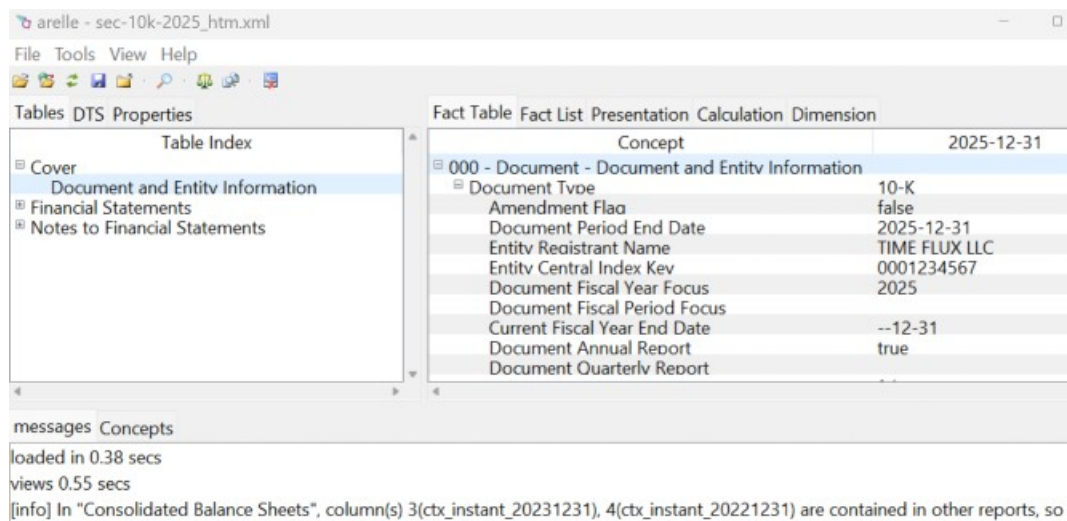
Under **View** → **Edgar Renderer**: select “Show filing data”, “Show redlining and redactions”, “Show table menu”, and “Validate before rendering”.

Under **Help** → **Manage plug-ins**: click on “Select” button, from the list choose “Edgar Renderer”, and click OK.

(Optional) Under **Help** → **Manage packages**: click on “Locally” button, from directory, C:\AppIO\user_input, choose “us-gaap-2026.zip”, and click OK.

Restart Arelle.

Now, to open a Form 10-K file: File → Open file → sec-10k-2025.htm → Open



Browser Directly

When the 10-K file, sec-10-k.htm, is opened directly with a browser, the iXBRL tagged elements are not highlighted. The embedded tags do not interfere with normal HTML viewing.

UNITED STATES
SECURITIES AND EXCHANGE COMMISSION
Washington, D.C. 20549

FORM 10-K

☒ ANNUAL REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES EXCHANGE

ACT OF 1934

For the fiscal year ended December 31, 2025

☐ TRANSITION REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES

EXCHANGE ACT OF 1934

For the transition period from _____ to _____

Commission File Number: 001-31123

TIME FLUX LLC

(Exact name of registrant as specified in its charter)

Maryland	31-1234567
(State or other jurisdiction of incorporation or organization)	(I.R.S. Employer Identification No.)

1 Technology Drive, Columbia, MD 21046

(Address of principal executive offices) (Zip Code)

Balance Sheets

	Year ended December 31, 2025	Year ended December 31, 2024
ASSETS		
Cash and Equivalents	\$81,075,905	\$59,475,456
Short-Term Investments	21,623,461	11,155,182
Accounts Receivable	30,271,873	20,540,624
Allowance for Doubtful Accounts	(5,636,960)	(3,417,720)
Inventory	4,200,000	2,940,000
Prepaid Expenses	2,857,907	1,748,600
Property, Plant & Equip	9,771,284	8,971,128
Accumulated Depreciation	(2,280,126)	(1,418,688)
Total Assets	141,883,344	99,994,581

The Arelle “Inline Viewer” web page displays the HTML file just like the SEC Filings, because Arelle adds orange border styles to highlight the iXBRL tagged items between two orange lines.

Balance Sheets

Sample Notes

Note 7 - Short-term and Long-term Debt

TIME FLUX LLC maintains a revolving credit facility and term loan arrangements to support its working capital needs and general corporate purposes. Borrowings under these facilities bear interest at prevailing market rates, which are subject to change based on current financial conditions and market indices. The debt is structured with monthly payment obligations.

These amounts reflect the balance sheet date's position and are subject to change based on future borrowing activities and repayments.

The revolving credit facility and term loans do not contain any specific covenants that would require disclosure here. Therefore, no further discussion of compliance with covenants is necessary.

Short-term debt outstanding was \$176,776, and long-term debt outstanding was \$3,885,494. The following table summarizes the Company's long-term debt by contractual maturity:

Fiscal Year	Principal Due
2026	\$1,403,343
2027	\$1,540,594
2028	\$567,909

Inline CSS

If you inspect one of the items with orange border, you will see some true false conditions, plus a border color, like the following:

```
[enabled-fact="true"][inline-block-fact="true"][selected-fact="false"]{outline:solid 2px #FF6600;}
```

When you load an SEC filing report into the interactive Arelle iXBRL Viewer, the viewer injects custom data attributes into the HTML. This specific CSS block handles how a data tag behaves visually when you interact with it:

- **[enabled-fact="true"]** : The element contains a valid, interactive iXBRL financial tag.
- **[inline-block-fact="true"]** : The tag is rendered structurally as an inline-block layout element.
- **[selected-fact="false"]** : The tag is currently active/visible on the page but is **not** the one you have currently clicked or highlighted.
- **{outline:solid 2px #FF6600;}** : Applies a solid, 2-pixel-thick orange outline (**#FF6600**) to any element that matches all three conditions at the same time. This lets you immediately spot unselected, yet active financial figures on the statement page.

If you inspect the web page near the top, you will see

html → head → body → iframe → html → head → style → body...

where “style” contains a list of conditions/css like the sample above, which apply to various parts of the filing. For example, in the Notes section, there is an orange border on the left side of each paragraph.

iXBRL Validation

In the lower half of the Arelle GUI, there are two tabs: messages and Concepts. We are interested only in the **messages** panel, which lists a few temporary files, plus any validation issues, such as:

Invalid linkbase name, must match "{base}-{yyyymmdd}_pre.xml": tifix-pre.xml

Calculation inconsistent ... reported sum 35,596,507 computed sum 57,640,378

To save the messages, got to Tools → **Messages log**: choose “Save to file”, and in the file dialog box, choose a log file.

Invalid or inconsistent data must be resolved during development validation testing, as described in the subsequent iXBRL section.

Rounding issue: if you see a message as below, where the difference is only 1 dollar, you may ignore it, since Arelle computed the value without the raw data.

[calc11e:inconsistentCalculationUsingRounding] Calculation inconsistent from us-gaap:NetIncomeLoss in link role http://www.tifx.com/20251231/role/IncomeStatement reported sum [28305785, 28305785] computed sum [28305786.0, 28305786.0] context ctx_dur_20250101_20250930 unit USD - sec-10q-2025-q3.htm 632, 639, 646, 662, 901, 903, 1020, 1023

Arelle References:

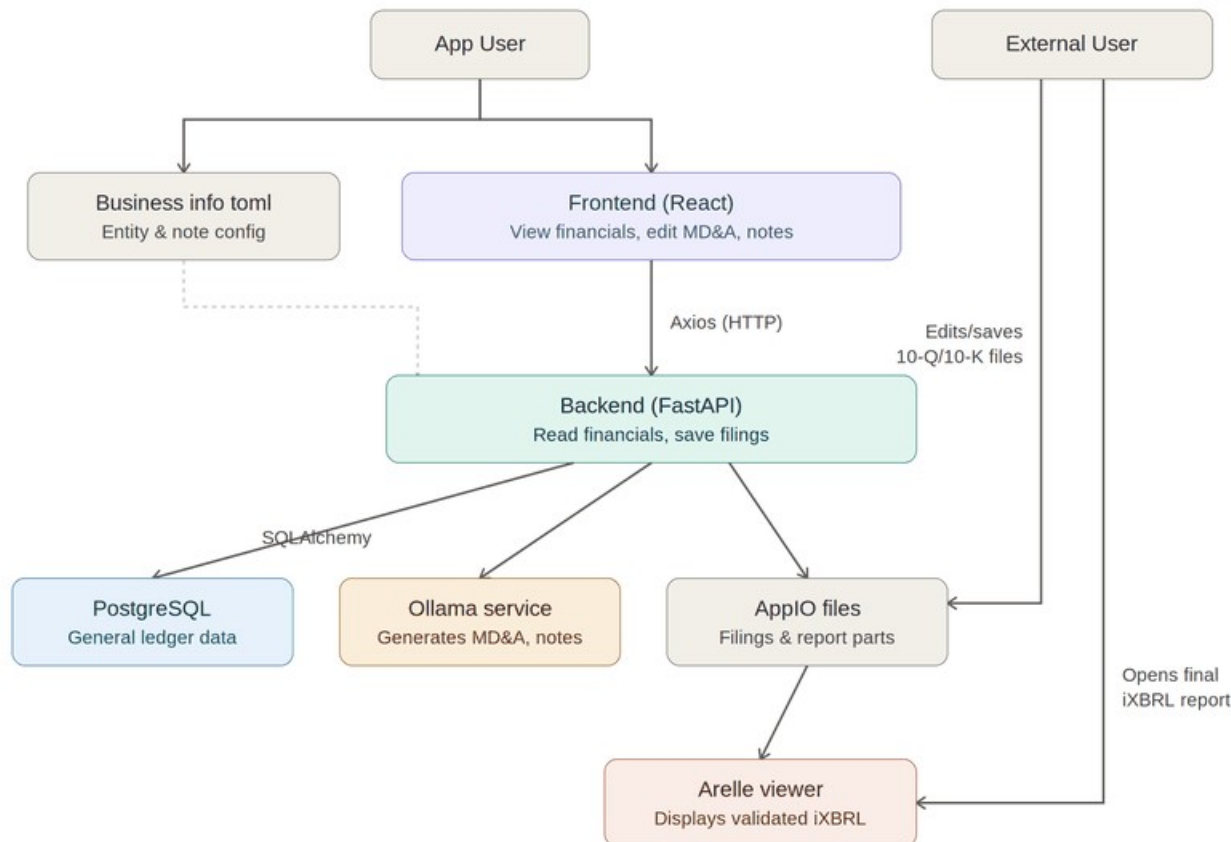
https://arelle.readthedocs.io/en/latest/command_line.html

<https://arelle.readthedocs.io/en/2.41.6/>

<https://arelle.org/arelle/pub/>

Section 4 - Architecture & Scope

Behind the application's clean and straightforward user interface lies a sophisticated data compliance engine designed to handle the heavy lifting of generating SEC Form 10-Q and 10-K reports. The system leverages a decoupled frontend and backend architecture. This separation ensures that complex data orchestration, financial calculations, and AI-driven agent workflows run seamlessly behind the scenes, far removed from the primary user view.



The AppUser manages business_info.toml directly and accesses the React frontend to review financial data and refine MD&A/Notes content. Front-end actions trigger Axios calls to a FastAPI backend. FastAPI reads financial records from PostgreSQL via SQLAlchemy, invokes Ollama to draft MD&A and Notes narratives, and outputs both single .docx files and assembled filings into AppIO. Simultaneously, External Users drop custom .docx components into AppIO and then open Arelle to validate and inspect the resulting iXBRL filing.

Core Repository Structure

The main codebase is cleanly separated into a Python backend, documents, and a modern React frontend.

```
/disclosure-pilot-ai/
├── backend/
│   ├── app/
│   │   ├── agents/      # AI agent implementations
│   │   ├── api/         # FastAPI route handlers & endpoints
│   │   ├── core/        # App configurations, security, & database setup
│   │   ├── models/      # User authentication database ORM model (SQLAlchemy)
│   │   ├── schemas/     # Input/output validation contracts (Pydantic)
│   │   └── services/     # Core business logic layer, database ORM models (SQLAlchemy)
│   └── tests/           # Backend unit & integration test suites
├── docs/                # Project description and setup
├── frontend/
│   ├── dist/            # index files
│   └── src/
│       ├── api/         # Axios clients & HTTP wrappers
│       ├── components/  # UI widgets: menus, tables, editors, dialogs
│       ├── hooks/       # User authentication context
│       ├── pages/       # Top-level views
│       ├── styles/      # Global CSS module
│       └── tests/       # Frontend UI & unit tests
```

Boot Sequence

Backend main.py → Frontend HomePage.jsx

The table below traces the full startup path of the application, from the FastAPI process launching in main.py through to an authenticated user landing on HomePage.jsx in the React frontend. The sequence spans two phases: (1) server/process startup, which happens once when the backend launches, and (2) the browser/client boot, which happens each time a user loads the page, culminating in an authentication check that decides whether they see the login page or the home page.

Boot Sequence Summary Table

Stage	File / Component	What happens
1. Server start	main.py	Uvicorn imports app.main:app; FastAPI instance is created and CORS middleware attached.
2. Config load	main.py / app.core.config	settings loaded (DB_HOST, OLLAMA_HOST/URL, WIN_APPIO_DIR, REPORTS_DIR, etc.); each is logged at startup.
3. Routers mounted	main.py	auth, report_10q, and report_10k routers registered under /api/auth, /api/10q, /api/10k — before the SPA catch-all so API paths always win.
4. Static assets mounted	main.py	If FRONTEND_DIST exists, /assets is mounted to serve the Vite JS/CSS bundles.
5. Browser requests "/"	main.py → index.html	serve_root() (or the SPA catch-all) returns index.html, which references the built JS bundle and CSS file.
6. React boots	main.jsx	createRoot(...).render(<App />) mounts the React tree into #root inside index.html.
7. Routing auth context set up	App.jsx	BrowserRouter wraps the app; AuthProvider (from useAuth.jsx) wraps everything so auth state is available app-wide.
8. Session check	useAuth.jsx	On mount, AuthProvider reads access_token from localStorage. If present, calls authApi.getMe() to fetch the user and validate the token; if absent, loading is set to false immediately with user = null.
9. Root route resolves	App.jsx (AuthRoute)	While loading is true, a "Loading..." placeholder is shown. Once resolved: if user exists, redirect to /home; otherwise render LoginPage.
10. Login form shown	LoginPage.jsx	User is presented the Sign In form (with toggles to Create Account / Forgot Password).
11. Credentials submitted	LoginPage.jsx → useAuth.jsx	handleSubmit() calls login(email, password), which POSTs to the backend auth endpoint, stores the returned access_token in localStorage, then calls getMe() again to populate user in context.

Stage	File / Component	What happens
12. Redirect to home	App.jsx (AuthRoute)	Because user is now set, AuthRoute re-renders and issues Navigate to /home.
13. Protected layout renders	ProtectedLayout → HomePage.jsx	The /home route renders inside the authenticated layout; HomePage.jsx reads user from useAuth() and displays the welcome message, including the signed-in user's email.

Data Workspace (AppIO)

Input and output data is stored separately from the application code, as listed below.

AppIO/

```

|— reports/
|— sec10k/
|— sec10q-q1/
|— sec10q-q2/
|— sec10q-q3/
|— user_input/

```

Before describing the report document layout, the user must understand the **user_input** files, listed here:

```

business_info.toml
item001-10-k-cover.docx
item001-10-q-cover.docx
note_list_10k.toml
note_list_10q-q1.toml
note_list_10q-q2.toml
note_list_10q-q3.toml
us-gaap-2026.zip

```

The **business_info.toml** file contains basic company information such as name, description, ticker, address, filer status, auditor, shares, etc. Some of this data is applied to the report cover page as substitution variables as seen in item001-10-k-cover.docx and item001-10-q-cover.docx. (**Notice:** the parameters “legal_proceedings” and “purchase_commitments” are special inputs for the “Commitments and Contingencies” note, to be used only when either of

these conditions become abnormal (see `notes_agent.py`, but no code change required). Rather than setting these parameters, the user may also just edit the note directly from the frontend.)

The notes list file, **`note_list_10k.toml`**, may be edited by the user each year to select which Notes to the Financial Statements to include in the 10-K report, by setting the “select” parameter to “true”. Similarly, the **`note_list_10q-qi`** files may be edited for each quarterly 10-Q report. **Notice:** when adding new notes, sometimes the Python code must be modified (see `notes_agent.py`, `notes_registry.py`, and `xbml_tagger.py`). Currently there are three placeholders for new notes in `notes_registry.py`: Stock-Based Compensation, Fair Value Measurements, Business Combinations. Use these placeholders as templates for adding more notes as desired.

The last file, **`us-gaap-2026.zip`**, is a convenience for the Arelle validation and viewer tool, to store taxonomy information locally rather than downloading from the internet.

API Endpoints

The Application Programming Interface (API) endpoints are summarized below for the connections between the frontend (`client.js`) and backend (`auth.py`, `report_10q.py`, and `report_10k.py`). See `schemas.py` for the request and response objects, such as `QuarterlyRequest` and `FinancialStatements`, for the 10-Q endpoint: `/api/10q/financials`.

Auth Endpoints

Method	Path	Description
POST	<code>/api/auth/register</code>	Create new user
POST	<code>/api/auth/login</code>	Login → JWT token
POST	<code>/api/auth/reset-password</code>	Reset password
GET	<code>/api/auth/me</code>	Current user info

10-Q Endpoints

Method	Path	Description
POST	/api/10q/financials	Get financial statements
POST	/api/10q/save-financials	Save item010-Financial-Statements.docx
POST	/api/10q/mda	Generate MD&A narrative
POST	/api/10q/save-mda	Save item020-MDA.docx
POST	/api/10q/notes	Generate Notes narrative
POST	/api/10q/save-notes	Save item011-Notes-to-Financial-Statements.docx
POST	/api/10q/generate-report	Merge → sec-10-q.docx
POST	/api/10q/generate-edgar-html	Convert to EDGAR HTML

10-K Endpoints

Method	Path	Description
POST	/api/10k/financials	Get annual financial statements
POST	/api/10k/save-financials	Save item081-Financial-Statements.docx
POST	/api/10k/mda	Generate annual MD&A
POST	/api/10k/save-mda	Save item070-MDA.docx
POST	/api/10k/notes	Generate annual Notes
POST	/api/10k/save-notes	Save item082-Notes-to-Financial-Statements.docx
POST	/api/10k/generate-report	Merge → sec-10-k.docx
POST	/api/10k/generate-edgar-html	Convert to EDGAR HTML

SQLAlchemy Database Interface

SQLAlchemy serves as the backend Object-Relational Mapper (ORM) and SQL toolkit, providing object-oriented access to the underlying SQL databases.

Module & Schema Mapping

- **Authentication:** auth.py accesses the **applogins** schema to handle user authorization (notably via get_user_by_email).
- **Financial Services (10-Q / 10-K):** financial_service.py accesses the **bookkeeper** schema. Key query functions include:
 - get_chart_of_accounts
 - get_loans
 - get_gl_balances_cumulative
 - get_gl_balances_range
 - _acct_bal_as_of

Data Serialization

User login responses are returned to the frontend in the AppUser class, specified in user.py.

Financial query results populate specific row-level data models in schemas.py (BalanceSheetRow, IncomeStatementRow, CashFlowRow, and StockholdersEquityRow). These are grouped as lists inside a parent FinancialStatements wrapper class and sent to the frontend as a structured payload.

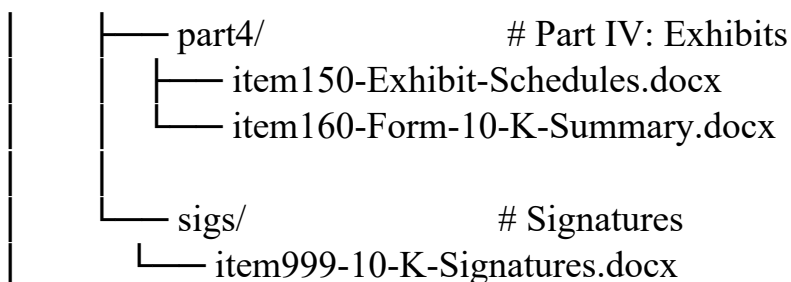
Document Generation Workspace (AppIO)

To successfully compile massive, highly regulated regulatory filings, the application uses a modular, multi-file assembly strategy. Each official item of an SEC Form 10-K is generated as an independent document snippet before being systematically compiled into the final master report. (The Form 10-Q files, not shown below, are listed under sec10q-q1, sec10q-q2, sec10q-q3.)

AppIO

```
|— reports/
|   |— sec-10q-2025-q1.docx      # Final Compiled 10-Q Q1 Report
|   |— sec-10q-2025-q2.docx      # Final Compiled 10-Q Q2 Report
```

sec-10q-2025-q3.docx	# Final Compiled 10-Q Q3 Report
sec-10k-2025.docx	# Final Compiled 10-K Report
sec10k/	# Granular 10-K Component Registry
intro/	# Document Preliminaries
item001-10-k-cover.docx	# Cover Page
item002-10-k-TOC.docx	# Table of Contents
part1/	# Part I: Operational Disclosures
item010-BusinessProfile.docx	
item01A-Risk-Factors.docx	
item01B-Unresolved-Staff-Comments.docx	
item01C-Cybersecurity.docx	
item020-Properties.docx	
item030-Legal-Proceedings.docx	
item040-Mine-Safety-Disclosures.docx	
part2/	# Part II: Financials, MD&A, and Notes
item050-Common-Equity.docx	
item060-Reserved.docx	
item070-MDA.docx	# Management's Discussion & Analysis
item07A-Disclose-Market-Risk.docx	
item080-AuditorReport.docx	
item081-Financial-Statements.docx	
item082-Notes-to-Financial-Statements.docx	
item090-Disagreements-with-Accountants.docx	
item09A-Controls-Procedures.docx	
item09B-Other-Information.docx	
item09C-Disclosure-Foreign-Inspections.docx	
part3/	# Part III: Corporate Governance
item100-Directors-Exec.docx	
item110-Exec-Compensation.docx	
item120-Beneficial-Owners.docx	
item130-Certain-Relationships.docx	
item140-Accountant-Fees.docx	



Architectural Design Choices

Even though the final Form 10-K (or Form 10-Q) report is submitted to the SEC as a single, consolidated file, the system deliberately processes individual sections as separate documents. This architectural choice was made to simplify application logic and support collaborative workflows:

- **Distributed Authoring:** Separate files allow different internal employees or outsourced teams to work on specific sections simultaneously without version control conflicts.
- **Third-Party Integration:** Certain sections are naturally generated by external software or specialized vendors. For instance, item080-AuditorReport.docx is typically prepared and delivered by an outside auditing firm.

Application Limitations

To maintain a highly focused project scope, the application's direct editing and generation capabilities are strictly limited.

Key Takeaway: The software only edits and produces six specific files (for 10-K):

1. item001-10-k-cover.docx (Cover Page)
2. item002-10-k-TOC.docx (Table of Contents)
3. item070-MDA.docx (Management's Discussion and Analysis)
4. item081-Financial-Statements.docx (Balance Sheet, Income Statement, Cash Flow Statement, Stockholders Equity)
5. item082-Notes-to-Financial-Statements.docx (Notes to Financial Statements)
6. sec-10k-2025.htm (Complete Filing with iXBRL)

Important: Create the remaining .docx item files outside of this application.

The primary function of the application is to compile and merge the entire collection of individual .docx item files into the final, cohesive Form 10-K or Form 10-Q report ready for submission to the SEC.

While report files are not overwritten, the many .docx item files are assumed to be overwritten in this application. Thus, to preserve these files from year to year, the user may zip them up in an archive.

Notice: This application is a proof-of-concept demonstration for generating 10-K and 10-Q filings, not an accounting platform.

- **Read-Only Operation:** The app never inserts, updates, or alters the underlying financial database.
- **Simplified Schema:** Designed for clarity rather than live operations, the database omits complex items like stock option compensation, other comprehensive income (foreign currency, pension liability), business acquisitions, deferred taxes, and closing entries.
- **Synthetic Data:** Historical data is AI-generated solely to allow users to simulate filing reports.
- **Limited Disclosures:** The backend implements only eight note types for the *Notes to Financial Statements*, whereas standard corporate 10-Ks typically include 15–20.

React Client references:

<https://thepythoncode.com/article/fullstack-notes-app-with-fastapi-and-reactjs>

<https://www.bairesdev.com/blog/react-spa-single-page-application/>

<https://www.geeksforgeeks.org/python/fastapi-templates/>

Fastapi Server references:

<https://realpython.com/fastapi-python-web-apis/>

<https://fastapitutorial.com/blog/serving-html-fastapi/>

<https://fastapi.tiangolo.com/reference/apirouter/#fastapi.APIRouter.get--example>

Section 5 - Database Schema

This section outlines the relational database architecture for the bookkeeper schema, hosted on a PostgreSQL instance. The tables are structured to ensure absolute data integrity, performance optimization, and auditability for financial transactions.

1. Architectural Overview

The schema comprises seven primary tables structured to manage a multi-currency double-entry journal ledger, operational sub-ledgers (Accounts Payable and Accounts Receivable), fixed asset tracking, depreciation logs, and master reference entities (Customers and Vendors). Transactional balances are strictly guarded at the database level via deferred constraints and system views.

2. Detailed Table Specifications

2.1 Table: bookkeeper.chart_of_acct

Master metadata directory defining the comprehensive structure of financial accounts and normal balance rules.

Column Name	Data Type	PK?	Description / Constraints
account_number	VARCHAR(10)	YES	Primary Key. Unique index code for the account.
account_name	VARCHAR(100)	NO	Standard financial title of the specific account.
account_type	VARCHAR(30)	NO	Top-level categorization. CHECK constraint ensures values match: 'Asset', 'Liability', 'Equity', 'Revenue', 'Expense'.
account_subtype	VARCHAR(30)	NO	Sub-classification layer for finer reporting granularities.
normal_balance	VARCHAR(6)	NO	Standard accounting balance type. CHECK constraint restricts to: 'Debit', 'Credit'.

2.2 Table: bookkeeper.vendors

Master reference table containing validated corporate credential details for suppliers and external vendors.

Column Name	Data Type	PK?	Description / Constraints
vendor_id	VARCHAR(15)	YES	Primary Key. Unique identifier for the vendor/supplier.
vendor_name	VARCHAR(50)	NO	Official trade or commercial entity name of the vendor.

2.3 Table: bookkeeper.customers

Master directory storing validated identity profiles for corporate, institutional, and individual commercial consumers.

Column Name	Data Type	PK?	Description / Constraints
customer_id	VARCHAR(15)	YES	Primary Key. Unique identifier for the client entity.
customer_name	VARCHAR(50)	NO	Legal commercial or personal name of the customer.

2.4 Table: bookkeeper.journal_entries

The central transaction ledger tracking document headers, publication dates, and validation state checkpoints.

Column Name	Data Type	PK?	Description / Constraints
id	BIGSERIAL	YES	Primary Key. Autoincrementing unique ledger entry identifier.
entry_date	DATE	NO	The explicit posting or execution date. Defaults to CURRENT_DATE.
description	TEXT	NO	Auditable narrative describing the business purpose of the transaction.
status	VARCHAR(15)	NO	Lifecycle state of the entry. Defaults to 'DRAFT'.
created_at	TIMESTAMP W/ TZ	NO	System record creation timestamp. Defaults to CURRENT_TIMESTAMP.

2.5 Table: bookkeeper.journal_lines

Atomic double-entry transaction fragments. Contains individual line-item allocations mapping debits and credits to target sub-accounts.

Column Name	Data Type	PK?	Description / Constraints
id	BIGSERIAL	YES	Primary Key. Autoincrementing identifier for the transaction line.
journal_entry_id	BIGINT	NO	Foreign Key reference to bookkeeper.journal_entries(id) ON DELETE CASCADE.
account_number	VARCHAR(10)	NO	Foreign Key reference linking lines to bookkeeper.chart_of_acct(account_number).
debit	NUMERIC(15, 4)	NO	Absolute debit amount. Must be non-negative. Defaults to 0.
credit	NUMERIC(15, 4)	NO	Absolute credit amount. Must be non-negative. Defaults to 0.
currency	VARCHAR(3)	NO	Three-letter ISO currency identifier. Defaults to 'USD'.

Constraint:

chk_debit_credit_exclusivity enforces that a line must have a positive Debit OR Credit, but not both.

2.6 Table: bookkeeper.accounts_payable

Tracks operational sub-ledger liabilities, procurement obligations, and audit trails to settlement records.

Column Name	Data Type	PK?	Description / Constraints
ap_id	VARCHAR(15)	YES	Primary Key. Unique identifier for the accounts payable record.
vendor_id	VARCHAR(15)	NO	Foreign Key reference to bookkeeper.vendors(vendor_id).

Column Name	Data Type	PK?	Description / Constraints
invoice_date	DATE	NO	The date the invoice was officially generated by the vendor.
due_date	DATE	NO	The contractually bound settlement deadline.
amount	NUMERIC(15, 4)	NO	The gross absolute financial liability value.
status	VARCHAR(15)	NO	Payment or processing state. CHECK: 'UNPAID', 'PAID', 'VOIDED'. Defaults to 'UNPAID'.
currency	VARCHAR(3)	NO	Three-letter ISO currency identifier. Defaults to 'USD'.
recognition_entry_id	BIGINT	NO	Foreign Key linking to the originating entry in bookkeeper.journal_entries.
settlement_entry_id	BIGINT	NO	Foreign Key linking to the final payment entry in bookkeeper.journal_entries.
			Constraint: chk_ap_payment_trail mandates settlement_entry_id must be populated if and only if status is 'PAID'.

2.7 Table: bookkeeper.accounts_receivable

Tracks active incoming credit claims and asset volumes due from customer consumer accounts.

Column Name	Data Type	PK?	Description / Constraints
ar_id	VARCHAR(15)	YES	Primary Key. Unique invoice claim identifier.
customer_id	VARCHAR(15)	NO	Foreign Key reference to bookkeeper.customers(customer_id).
invoice_date	DATE	NO	The date the commercial invoice was generated and issued.

Column Name	Data Type	PK?	Description / Constraints
due_date	DATE	NO	The contractual settlement deadline for the customer.
amount	NUMERIC(15, 4)	NO	The gross absolute financial asset volume claimable.
status	VARCHAR(15)	NO	Collection state. CHECK: 'UNPAID', 'PAID', 'VOIDED', 'BAD_DEBT'. Defaults to 'UNPAID'.
currency	VARCHAR(3)	NO	Three-letter ISO currency identifier. Defaults to 'USD'.
recognition_entry_id	BIGINT	NO	Foreign Key linking to the revenue recognition entry in bookkeeper.journal_entries.
settlement_entry_id	BIGINT	NO	Foreign Key linking to the customer payment entry in bookkeeper.journal_entries.
			Constraint: chk_ar_payment_trail mandates settlement_entry_id must be populated if and only if status is 'PAID'.

2.8 Table: bookkeeper.assets

Maintains an active operational inventory of capitalized objects and long-term tangible corporate assets.

Column Name	Data Type	PK?	Description / Constraints
asset_id	VARCHAR(15)	YES	Primary Key. Unique identifier for the fixed asset item.
asset_name	VARCHAR(100)	NO	Descriptive commercial name of the capitalized asset.
purchase_date	DATE	NO	Acquisition date for accounting tracking.
cost	NUMERIC(15, 4)	NO	Gross historical acquisition or production cost volume.

Column Name	Data Type	PK?	Description / Constraints
service_years	INTEGER	NO	Estimated useful operational lifetime projection in years.
dept	VARCHAR(15)	NO	Cost center allocation or organizational department tag.
currency	VARCHAR(3)	NO	Three-letter ISO currency identifier. Defaults to 'USD'.

2.9 Table: bookkeeper.depreciation

Records recurring systematic book-value write-downs of assets across operational fiscal cycles.

Column Name	Data Type	PK?	Description / Constraints
depreciation_id	VARCHAR(15)	YES	Primary Key. Unique depreciation ledger row index.
asset_id	VARCHAR(15)	NO	Foreign Key reference link matching target bookkeeper.assets(asset_id).
year	INTEGER	NO	The operational fiscal year of the logged depreciation expense.
annual_depreciation	NUMERIC(15, 4)	NO	The absolute value expensed as depreciation during the specified year.
accumulated_thru_year	NUMERIC(15, 4)	NO	Total cumulative depreciation written off up to the logged year.
currency	VARCHAR(3)	NO	Three-letter ISO currency identifier. Defaults to 'USD'.

3. Database Views & Performance Layer

3.1 View: bookkeeper.general_ledger

Acts as an auditable public transaction log exposing only finalized operations. It automatically resolves and joins transaction headers and detailed lines to report posted journals. It includes line items if and only if the header status is explicitly set to 'POSTED'.

3.2 Performance Indexes

To facilitate high-throughput reporting, sub-second queries, and instant reconciliation reviews, the database implements specific relational performance indexing paths:

- `idx_journal_lines_entry`: Fast lookup index on `journal_lines(journal_entry_id)` to accelerate transaction construction.
- `idx_journal_lines_account`: Optimization index on `journal_lines(account_number)` ensuring rapid sub-account evaluations.
- `idx_journal_entries_date_posted`: A partial conditional index on `journal_entries(entry_date)` filtered WHERE status = 'POSTED' to optimize general ledger views.

4. Business Rules & Operational Constraints

Double-Entry Integrity: Strict transactional balance is protected via a row-by-row procedural trigger (`bookkeeper.enforce_journal_entry_balance()`). The constraint trigger `trigger_enforce_balance` acts AFTER INSERT, UPDATE, or DELETE on any line-item. It is defined as DEFERRABLE INITIALLY DEFERRED, meaning the PostgreSQL engine permits temporary transaction imbalances while compiling line-items, but verifies and strictly throws a rollback exception if the aggregate Debits minus Credits do not sum to exactly zero at commit execution.

Line-Level Mutual Exclusivity: Every transaction fragment row in `journal_lines` must have a single clear financial direction. The CHECK constraint `chk_debit_credit_exclusivity` guarantees that either `debit > 0` and `credit = 0`, or `credit > 0` and `debit = 0`. Negative financial value logs are completely blocked.

Currency Uniformity: Across all sub-ledgers, transaction logs, and primary ledgers (`journal_lines`, `accounts_payable`, `accounts_receivable`, `assets`, and `depreciation`), a standard string default value of 'USD' is natively applied if explicit ISO currency codes are missing.

Sub-Ledger Paper Trails: Both Accounts Payable and Accounts Receivable tightly bind operational invoicing to general journal lines through recognition and settlement links. If an invoice status transitions to 'PAID', database constraints mandate that the settlement journal entry identifier must be populated with a valid transaction reference, providing a pristine audit trail from operations directly to financial reports.

Section 6 - Accounting Fundamentals

1. Overview & Architectural Context

This section describes the financial accounting logic for the financial application. To maintain strict precision and auditing integrity, the application uses a double-entry bookkeeping architecture built on top of a relational PostgreSQL schema (bookkeeper). Every economic event impacting the organization is recorded atomically across balanced debits and credits, creating an unalterable audit trail that serves as the single source of truth for downstream financial reporting.

2. Core Ledger Database Schema & Validation Constraints

The application ledger relies on two central tables to track transactions:

`bookkeeper.journal_entries` and `bookkeeper.journal_lines`. These mirror the traditional accounting concept of a 'Journal Entry' and its corresponding 'Journal Lines.'

- **bookkeeper.journal_entries:** records the transaction-level metadata, including the `entry_date`, text description, and transaction status (e.g., DRAFT, POSTED).
- **bookkeeper.journal_lines:** stores individual debits and credits. Each line enforces mutual exclusivity via the `chk_debit_credit_exclusivity` constraint, ensuring that a single line contains either a debit or a credit, but never both.

DATABASE INTEGRITY CONSTRAINT: To guarantee that the fundamental accounting equation ($\text{Assets} = \text{Liabilities} + \text{Equity}$) remains in perpetual balance, the database enforces a transactional constraint. An AFTER INSERT OR UPDATE OR DELETE constraint trigger (`trigger_enforce_balance`) executes the PL/pgSQL function `bookkeeper.enforce_journal_entry_balance()`. This function aggregates total debits minus total credits for the modified `journal_entry_id`. If the net balance does not equal zero, the database rolls back the transaction and throws a fatal exception.

What is the Chart of Accounts?

The chart of accounts (COA) organizes financial information into five core account types (or categories)—assets, liabilities, equity, revenue, and expenses—typically using a standardized numeric coding system to establish a clear reporting hierarchy. In essence, the COA acts as the master data model that translates raw, day-to-day operational transactions into the clean, consolidated **line items** that ultimately populate your **balance sheet**, **income statement**, and 10-K reports. See database table, `bookkeeper.chart_of_acct`. For each of the five account types, there are a few subtypes, as listed below.

Account number	Account name	Account type	Account subtype	Normal balance
1000	Cash and Equivalents	Asset	Current Asset	Debit
1050	Short-Term Investments	Asset	Current Asset	Debit
1100	Accounts Receivable	Asset	Current Asset	Debit
1150	Allowance for Doubtful Accounts	Asset	Contra Asset	Credit
1200	Inventory	Asset	Current Asset	Debit
1300	Prepaid Expenses	Asset	Current Asset	Debit
1500	Property, Plant & Equip	Asset	Fixed Asset	Debit
1550	Accumulated Depreciation	Asset	Contra Asset	Credit
2000	Accounts Payable	Liability	Current Liability	Credit
2100	Accrued Liabilities	Liability	Current Liability	Credit
2200	Short-term Debt	Liability	Current Liability	Credit
2300	Unearned Revenue	Liability	Current Liability	Credit
2500	Long-term Debt	Liability	Long-term Liability	Credit
3000	Common Stock	Equity	Paid-in Capital	Credit
3100	Retained Earnings	Equity	Retained Earnings	Credit
3200	Treasury Stock	Equity	Contra Equity	Debit
3300	Dividends Paid	Equity	Contra Equity	Debit
4000	Hardware Sales	Revenue	Product Revenue	Credit

Account number	Account name	Account type	Account subtype	Normal balance
4100	Software Sales	Revenue	Product Revenue	Credit
4200	Consulting	Revenue	Service Revenue	Credit
4300	Interest Income	Revenue	Non-Operating Revenue	Credit
5000	Cost of Goods Sold	Expense	Operating Expense	Debit
6000	Salaries and Wages	Expense	Operating Expense	Debit
6100	Supplies Expense	Expense	Operating Expense	Debit
6200	Lease Expense	Expense	Operating Expense	Debit
6300	Bad Debt Expense	Expense	Operating Expense	Debit
6400	Marketing Expense	Expense	Operating Expense	Debit
6500	R&D Expense	Expense	Operating Expense	Debit
6600	Depreciation Expense	Expense	Operating Expense	Debit
6700	Legal & Professional Fees	Expense	Operating Expense	Debit
7000	Interest Expense	Expense	Non-Operating Expense	Debit
8000	Income Tax Expense	Expense	Tax Expense	Debit

In addition, there is a “**normal balance**” that indicates the side, debit or credit, where **increases** are recorded. Understanding this removes confusion around debits and credits. Note that both credits and debits are allowed, the normal one is for increases, the opposite one is for decreases.

Debit Normal Balances

- **Assets:** Resources owned by the business (e.g., cash, equipment); increased with a debit, decreased with a credit.
- **Expenses:** Costs to run the business (e.g., rent, utilities); increased with a debit, decreased with a credit.

- **Dividends / Drawings:** Money taken out by owners; increased with a debit, decreased with a credit.

Credit Normal Balances

- **Liabilities:** Debts owed to others (e.g., loans, accounts payable); increased with a credit, decreased with a debit.
- **Equity:** Owner or shareholder stake in the company; increased with a credit, decreased with a debit.
- **Revenue (Income):** Money earned from sales or services; increased with a credit, decreased with a debit.

Contra Accounts

- **Opposite Balance:** These accounts reduce a parent account and carry the opposite normal balance (e.g., *Accumulated Depreciation* is a contra-asset with a normal credit balance).
- **Error Detection:** If an account shows the wrong balance type (like cash showing a credit), it usually signals a recording mistake.

Step-by-Step Transaction Examples & Ledger Representation

The following seven operational examples illustrate how transactional workflows populate the bookkeeper schema tables. For consistency, all entries are modeled with a status of POSTED, making them visible to the bookkeeper.general_ledger view.

Example 1: Application Initialization via Common Stock Sale

Scenario: The application is seeded by issuing common stock to founders for \$25,000,000 in cash. To immediately fund production, \$5,000,000 of that cash is allocated to establish starting raw materials and finished inventory, leaving \$20,000,000 as liquid cash.

Account Number	Account Name	Account Type	Debit (DR)	Credit (CR)
1000	Cash and Equivalents	Asset (Current)	\$20,000,000.00	\$0.00
1200	Inventory	Asset (Current)	\$5,000,000.00	\$0.00

3000	Common Stock	Equity (Paid-in Capital)	\$0.00	\$25,000,000.00
TOTAL			\$25,000,000.00	\$25,000,000.00

Example 2: Capital Asset Acquisition via Long-Term Debt

Scenario: The company secures corporate facilities and manufacturing equipment, purchasing Property, Plant & Equipment (PP&E) for \$10,000,000, fully financed via a long-term bank loan. This instantiates rows in the bookkeeper.assets and bookkeeper.loans operational tables, linked directly to the journal ledger.

Account Number	Account Name	Account Type	Debit (DR)	Credit (CR)
1500	Property, Plant & Equip	Asset (Fixed)	\$10,000,000.00	\$0.00
2500	Long-term Debt	Liability (Long-term)	\$0.00	\$10,000,000.00
TOTAL			\$10,000,000.00	\$10,000,000.00

Example 3: Hardware Product Sale (On-Credit)

Scenario: The company fulfills a bulk order of physical units for an enterprise client. The hardware is sold on open account terms for \$1,500,000, creating an Accounts Receivable asset and populating the sub-ledger bookkeeper.accounts_receivable.

Account Number	Account Name	Account Type	Debit (DR)	Credit (CR)
1100	Accounts Receivable	Asset (Current)	\$1,500,000.00	\$0.00
4000	Hardware Sales	Revenue (Product)	\$0.00	\$1,500,000.00
TOTAL			\$1,500,000.00	\$1,500,000.00

Example 4: Software Subscription Sale (Cash Settlement)

Scenario: Enterprise clients purchase cloud software licenses totaling \$2,500,000, processed through an electronic gateway and deposited instantly into the company's operating cash account.

Account Number	Account Name	Account Type	Debit (DR)	Credit (CR)
1000	Cash and Equivalents	Asset (Current)	\$2,500,000.00	\$0.00
4100	Software Sales	Revenue (Product)	\$0.00	\$2,500,000.00
TOTAL			\$2,500,000.00	\$2,500,000.00

Example 5: Cost of Goods Sold (COGS) Recognition

Scenario: To match the revenue recorded in Example 3 with its direct economic cost, the application reduces asset inventory and records an expense for the historical production cost of the items shipped, which totals \$800,000.

Account Number	Account Name	Account Type	Debit (DR)	Credit (CR)
5000	Cost of Goods Sold	Expense (Operating)	\$800,000.00	\$0.00
1200	Inventory	Asset (Current)	\$0.00	\$800,000.00
TOTAL			\$800,000.00	\$800,000.00

Example 6: SG&A Expense Disbursal (Salaries and Wages)

Scenario: Operational overhead representing Selling, General, and Administrative (SG&A) expenses is recorded. For this period, the company processes payroll disbursements for administrative and sales staff totaling \$700,000, settled out of operating cash.

Account Number	Account Name	Account Type	Debit (DR)	Credit (CR)
6000	Salaries and	Expense	\$700,000.00	\$0.00

	Wages	(Operating)		
1000	Cash and Equivalents	Asset (Current)	\$0.00	\$700,000.00
TOTAL			\$700,000.00	\$700,000.00

Example 7: Corporate Income Tax Expense Accrual

Scenario: Based on preliminary quarterly operating performance, the company computes its corporate tax provision. It recognizes \$300,000 in tax expense, deferred as an outstanding short-term obligation under accrued liabilities until the formal tax filing date.

Account Number	Account Name	Account Type	Debit (DR)	Credit (CR)
8000	Income Tax Expense	Expense (Tax)	\$300,000.00	\$0.00
2100	Accrued Liabilities	Liability (Current)	\$0.00	\$300,000.00
TOTAL			\$300,000.00	\$300,000.00

4. Database Row State Representation

The following tables demonstrate how the database stores the relational state inside `journal_entries` and `journal_lines` for the seven transactions recorded above.

Table: `bookkeeper.journal_entries`

id	entry_date	description	status
1	2026-06-17	Initial seed transaction via founder common stock sale	POSTED
2	2026-06-17	Acquisition of manufacturing equipment and facilities via loan	POSTED
3	2026-06-17	Bulk commercial hardware sales on open account terms	POSTED
4	2026-06-17	SaaS infrastructure customer annual renewal billing	POSTED
5	2026-06-17	Cost of goods sold matching recognition for hardware	POSTED

delivery

6	2026-06-17	Bi-weekly operational corporate payroll disbursement	POSTED
7	2026-06-17	Estimated quarterly corporate tax provision accrual	POSTED

Table: bookkeeper.journal_lines

id	journal_entry_id	account_number	debit	credit
1	1	1000	20000000.0000	0.0000
2	1	1200	5000000.0000	0.0000
3	1	3000	0.0000	25000000.0000
4	2	1500	10000000.0000	0.0000
5	2	2500	0.0000	10000000.0000
6	3	1100	1500000.0000	0.0000
7	3	4000	0.0000	1500000.0000
8	4	1000	2500000.0000	0.0000
9	4	4100	0.0000	2500000.0000
10	5	5000	800000.0000	0.0000
11	5	1200	0.0000	800000.0000
12	6	6000	700000.0000	0.0000
13	6	1000	0.0000	700000.0000
14	7	8000	300000.0000	0.0000
15	7	2100	0.0000	300000.0000

5. Generation of Financial Statements

At the close of an accounting reporting cycle, the application queries the unified general ledger database view (bookkeeper.general_ledger) to aggregate row-level debit and credit activity. These net balances generate the organization's primary financial reports.

The Income Statement

The Income Statement details the financial performance of the entity over a specific duration of time. It isolates operating and non-operating performance metrics by evaluating the core equation: $\text{Net Income} = \text{Revenue} - \text{Expenses}$

Income Statement Line Item	Amount (\$)	GL Source Accounts Involved
Hardware Sales Revenue	1,500,000.00	4000 - Hardware Sales (Example 3)
Software Sales Revenue	2,500,000.00	4100 - Software Sales (Example 4)
Total Revenue	4,000,000.00	Subtotal of Revenue Accounts
Cost of Goods Sold (COGS)	(800,000.00)	5000 - Cost of Goods Sold (Example 5)
Gross Profit	3,200,000.00	Revenue minus Direct Product Costs
Selling, General & Administrative (SG&A)	(700,000.00)	6000 - Salaries and Wages (Example 6)
Income Tax Expense	(300,000.00)	8000 - Income Tax Expense (Example 7)
Net Income	2,200,000.00	The 'Bottom Line' Transferred to Equity

THE ACCOUNTING BRIDGE: Net Income does not remain standalone. It serves as a continuous bridge that updates the entity's financial positioning on the Balance Sheet. During the closing entry process, the net earnings balance of \$2,200,000 shifts from the temporary income statement accounts into the permanent equity account 3100 - Retained Earnings under Equity, expanding the cumulative capital value held by shareholders.

The Balance Sheet

The Balance Sheet provides a structural snapshot showing the exact financial position of the company at a definitive point in time. It measures what the firm owns versus what it owes, validating the foundational identity: $\text{Assets} = \text{Liabilities} + \text{Shareholders' Equity}$

Balance Sheet Classification & Line Items	Net GL Balance (\$)	Derivation Mechanics & Account Activity Source
ASSETS		
Cash and Equivalents	21,800,000.00	1000: \$20,000,000 (Ex 1) + \$2,500,000 (Ex 4) - \$700,000 (Ex 6)
Accounts Receivable	1,500,000.00	1100: \$1,500,000 (Ex 3) outstanding commercial invoices
Inventory	4,200,000.00	1200: \$5,000,000 starting allocation (Ex 1) - \$800,000 COGS (Ex 5)
Property, Plant & Equip	10,000,000.00	1500: Fixed manufacturing plant acquisition value (Ex 2)
TOTAL ASSETS	37,500,000.00	Sum of all corporate resources under management
LIABILITIES		
Accrued Liabilities	300,000.00	2100: Estimated quarterly corporate tax obligation (Ex 7)
Long-term Debt	10,000,000.00	2500: Outstanding bank loan principal for capital asset (Ex 2)
TOTAL LIABILITIES	10,300,000.00	Sum of all outstanding external legal claims
EQUITY		
Common Stock	25,000,000.00	3000: Par value capital funding injected during initialization (Ex 1)
Retained Earnings	2,200,000.00	3100: Retained profit bridged directly from the Net Income pool
TOTAL EQUITY	27,200,000.00	Total net internal equity value held by corporate owners
TOTAL LIABILITIES & EQUITY	37,500,000.00	Verification: Liabilities (\$10.3M) + Equity (\$27.2M) = \$37.5M

Verification: Liabilities (\$10.3M) + Equity (\$27.2M) = Assets (\$37.5M)

References to Financial Accounting:

<https://medium.com/@abdulazizalthobaiti96/creating-an-accounting-information-system-part-1-the-journal-58aff3cd8f11>

<https://databasesample.com/database/accounting-ledger-database>

<https://www.pacificabs.com/knowledge-center/blog/small-business-accounting-essential-chart-of-accounts-every-owner-should-know/>

<https://www.ispirer.com/blog/best-database-for-financial-accounting-apps>

<https://www.hyperbots.com/glossary/audit-log-finance>

<https://onspring.com/resources/blog/what-is-an-audit-trail/>

Section 7 - LLM Analysis

1. How Large Language Models Work

Prompt-based AI relies entirely on Large Language Models (LLMs). To convert a user's prompt—such as turning software design specifications into functional Python code—the model relies on a process called **training**.

During training, the model's millions of internal weights are adjusted over multiple iterations until an optimal goal is met. This specific training methodology is called **Reinforcement Learning from Human Feedback (RLHF)**, which relies on human evaluation of a dedicated training dataset.

Mathematically, the core of LLM processing involves:

- **Matrix multiplications** to compute outputs.
- **Partial derivatives** for backpropagation, which mathematically updates the model's weights.

Once training is complete, these weights, which represent complex numerical relationships between words, are saved as an **LLM model** for the next phase called **inference**. When you input a new prompt, the software uses these optimized weights to compute a response, seamlessly turning design specs into code. Note, there is no backpropagation or weight updates in the inference phase.

2. Debunking LLM Terminology

Despite how impressive the output looks, it is important to clarify what LLM software actually is—and what it isn't.

- **Is it intelligent?** No.
- **Does it learn or reason?** No.
- **Does it understand English or your specific questions?** No.
- **Does it always produce correct results?** No.

The Library Analogy

If LLMs don't understand or reason, how do they generate acceptable responses? The answer is simple: **by lookup!**

Imagine a vast library containing every publication ever written. When you submit a request, a librarian looks up the absolute best set of documents that matches your prompt.

An LLM operates just like this library lookup process, only it is executed via matrix algebra using millions of model weights. Because it is fundamentally a lookup system, **if a concept was never published, the AI cannot know anything about it.**

3. Minimizing Mistakes and Hallucinations

Because LLMs absolutely require existing source material to look up, they will make mistakes or invent meaningless responses—known as **hallucinations**—when they run out of accurate references.

To counter this limitation, LLM software can use a technique called **Retrieval-Augmented Generation (RAG)**. Instead of relying strictly on static, pre-trained knowledge, the RAG feature allows the LLM to dynamically access external documents, which significantly helps the model avoid hallucinations.

LLM References:

<https://medium.com/@amallya0523/how-an-llm-understands-input-the-math-under-the-hood-114ac69f96c6>

<https://poloclub.github.io/transformer-explainer/>

<https://www.understandingai.org/p/large-language-models-explained-with>

<https://medium.com/@atifhussain/dont-get-fooled-by-llm-hype-why-understanding-goes-beyond-tutorials-7c15ee154476>

<https://www.gilesthomas.com/2025/09/maths-for-llms>

<https://medium.com/@anjalitanikella/understanding-the-math-behind-llm-models-and-fine-tuning-them-ee4ca222823f>

<https://medium.com/@kiplangatkorir/mathematical-foundations-of-large-language-models-541b196ccf84>

LLM hallucination:

<https://www.cnn.com/2026/04/23/business/ai-hallucination-sullivan-cromwell-nightcap>

Section 8: AI Agent Tutorial

An AI agent is not just a chatbot that answers a single question. It is a program that takes a goal, holds state, makes decisions, and performs a sequence of actions to reach that goal. The application implements two AI agents, both using the local version of ChatOllama LLM. One agent produces the Management's Discussion and Analysis (MD&A), while the other produces Notes to Financial Statements (Notes). In the application's MDA function (`mda_agent.py`), the agent uses a `ReportState` that flows through six specialized nodes. It is designed to act like a senior financial analyst: it receives raw financial data and produces a formal MD&A item for an SEC 10-Q or 10-K filing. Similarly, the Notes function (`notes_agent.py`), is designed like the MDA function, and is not described below.

Benefit

Modularity: each node does one job (validate, compute, enrich, generate, review).

Accuracy: Python calculates numbers before the LLM sees them.

Compliance: a dedicated review node checks SEC language.

Traceability: you can log state after any node.

Graph-Based Agent Networks: Graphs, Nodes, and Edges

Modern frameworks model agents as computational graphs, such as **LangGraph**, **GPTSwarm**, **Burr**, and **Mastra**, where agent behaviors are represented as interconnected nodes and edges that coordinate reasoning, tool use, and control flow. This framework provides clear architectural control over the agent's behavior. The graph manages the shared state, a sequentially updated data payload, that is passed systematically from one step to the next. A Node represents a discrete unit of computation within the graph.

Each node performs a specific operation on the current state and returns an updated state.

Examples of nodes include an LLM calling a specific prompt, an execution block running a SQL query, or a data validation function. An Edge defines the transition pathway between nodes.

Edges connect the output of one node to the input of another. There are two primary types of edges: Standard Edges, which dictate a sequential transition from Node A to Node B, and Conditional Edges, which evaluate the current state to dynamically route the execution path to different nodes based on some criteria. If a conditional edge is reached, the data is evaluated to decide if the state should loop backward (e.g., for error revision) or advance forward.

How to Code It

The MDA agent's workflow is built as follows:

```
workflow = StateGraph(ReportState)
workflow.add_node("analyze_financials", analyze_financials_node)
workflow.add_node("validate_financials", validate_financials_node)
workflow.add_node("compute_kpis", compute_kpis_node)
workflow.add_node("enrich_context", enrich_context_node)
workflow.add_node("generate_mda", generate_mda_node)
workflow.add_node("compliance_review", compliance_review_node)

workflow.set_entry_point("analyze_financials")
workflow.add_edge("analyze_financials", "validate_financials")
workflow.add_edge("validate_financials", "compute_kpis")
workflow.add_edge("compute_kpis", "enrich_context")
workflow.add_edge("enrich_context", "generate_mda")
workflow.add_edge("generate_mda", "compliance_review")
workflow.add_edge("compliance_review", END)
```

The Six Nodes

analyze_financials: Node 1 – initializes ReportState, sets status="analyzing", runs the three `_summarize_*` helpers to create `balance_sheet_summary`, `income_summary`, `cash_flow_summary`.

validate_financials: Node 2 – checks that Total Assets $\approx$ Liabilities + Equity, verifies prior-period data exists, and records any validation_issues.

compute_kpis: Node 3 – calculates deterministic metrics (YoY revenue growth %, net margin %, operating cash margin) and stores them as `kpi_summary`.

enrich_context: Node 4 – injects `business_description`, key initiatives, and acquisition details so the LLM does not create bracketed placeholders.

generate_mda: Node 5 – builds the system prompt ("senior financial analyst...") and user prompt with summaries and KPIs, for each MD&A subsection (e.g. Overview of Business and Highlights, Results of Operations, Liquidity and Capital Resources, etc.). Then ChatOllama is called, `llm.invoke()`, in a loop over each MD&A subsection. The loop is necessary to avoid degraded narratives when token limits are reached if all subsections are combined into one prompt.

compliance_review: Node 6 – runs a second LLM pass to ensure Forward-Looking Statements disclaimer, Liquidity and Capital Resources headings, and factual tone are present.

Input

The MDA agent accepts financial data as input, which include balance sheet, income statement, cash flow, and time periods. See schemas.py: BalanceSheetRow, IncomeStatementRow, CashFlowRow.

What the LLM Does

The Node 5 LLM, generate_mda, does not store sentences. It creates them at runtime because it:

Recognizes the genre — it's been trained on thousands of real 10-K MD&As, so it knows the SEC boilerplate ("Forward-Looking Statements disclaimer," "Liquidity and Capital Resources", etc.).

Fills the outline — it takes built-in headings and expands each one into paragraphs, weaving in the numbers passed to it (e.g. \$70M revenue, \$3.4M operating cash).

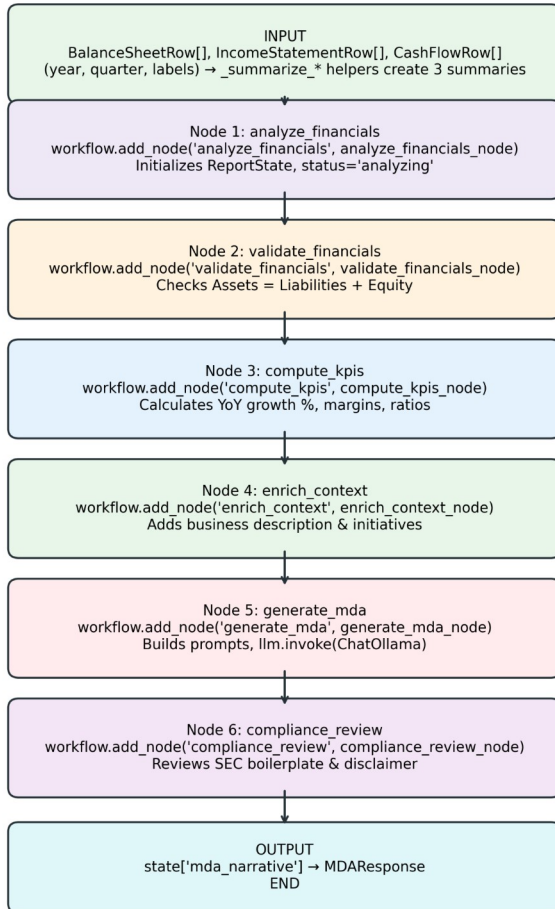
Output

After node 6, compliance_review, the state["mda_narrative"] contains the final MD&A text, returned as the MDAResponse(narrative, validation_warnings, ...), which the user may review and save to a file. If the response is a warning, the user can then check the financial data for any discrepancies.

Node Diagram

Visual reference for the workflow.add_node:

mda_agent.py - 6-Node LangGraph Workflow



AI Agent References:

<https://alain-airom.medium.com/pros-and-cons-using-containerized-ollama-vs-local-setup-d9bdf225bbb5>

<https://www.langchain.com/langgraph>

<https://github.com/metauto-ai/GPTSwarm>

<https://burr.apache.org/>

<https://mastra.ai/>

Section 9 - Data Generation Prompting

This section outlines the methodology for leveraging AI to generate realistic sample data for the PostgreSQL database. Keep in mind the code described here is temporary. Once the dataset is generated, the code is no longer required. In a real business environment, the database would already exist, and there would be no need for any simulated data.

The Data Generation Pipeline

A Python script, **build-accounting-workbook.py**, developed in collaboration with **Meta AI**, was used to generate an Excel workbook containing data sheets for each of the nine database tables. Once the workbook was generated, each sheet was converted into a CSV file (by Meta AI) and imported manually into the database. The complete detailed prompt can be found in **meta-ai-instructions.docx**, which serves as the master document referencing all dependent files.

Source Documentation & Inputs

To ensure the simulated data accurately reflected realistic business operations over a four-year period, several reference files were prepared prior to prompting the AI:

- **Database Specification:** Detailed the exact structural schema for all nine database tables.
- **Asset Ledger:** Listed every business asset alongside its name, cost, purchase date, and useful service life.
- **Core Instructions:** Defined the foundational requirements for generating financial accounting data compliant with 10-Q and 10-K reporting standards.

The Iterative Refinement Process

Generating high-quality, coherent financial data is inherently iterative and requires a significant amount of patience to master. Over multiple trials, the instructions were continuously updated as different data constraints became apparent:

- **Financial Guardrails:** Predefining annual revenue and expense targets was critical; without them, the AI consistently generated net losses year over year.
- **Asset Variety:** Providing an explicit asset file prevented the AI from generating an unrealistically small number of assets.

- **Transaction Logic:** The AI defaulted to cash payments unless explicitly instructed to simulate credit transactions.
- **Scope Management:** A careful balance was required; overloading the AI with too much detail simultaneously caused it to fail to generate any data at all.

AI Capabilities & Observations

One notable behavior of Meta AI was its ability to actively engage with the prompts rather than just passively reading them. Throughout the process, the AI asked clarifying questions to improve the dataset's integrity.

Example: During the generation process, Meta AI asked: “*Would you like to generate the balance sheets, income statement, and cash flow?*”

This was not part of the initial prompt requirements. It demonstrated the AI's inherent understanding of accounting principles, showing it could autonomously transform raw ledger information into structured financial statements without needing explicit procedural instructions.

Unfortunately, the instructions prompt did not specify using python class structures. All code was written within a single file. Because it was only meant to generate a one-off dataset, and because it worked well, there was no compelling reason to refactor. Thus, the resulting code is not as readable as the application code, discussed in the next section.

Section 10 – Powershell Script Prompting

Originally, this application used a Podman container, which worked well, but seemed to be overly complex for a Windows desktop application. Claude was prompted to replace the Podman container with some simple Powershell scripts. Then, Gemini was prompted to explain those scripts to users unfamiliar with Powershell.

Claude Prompt to create Powershell scripts

Podman refactoring question

Podman only runs on Linux. Podman has complex config files.

We have a Windows based application that uses an external database, Postgresql, an external LLM, ollama, an external xbrl validator, Arelle, and external IO files, AppIO.

I was wondering why we are using Podman?

Our application code is basically React and FastApi.

Why do we need a special packaging framework?

What about a simple zip (or equivalent) file, without any special framework?

Can we eliminate podman completely, and use Powershell scripts to package, install, and run the application?

The Powershell scripts

Claude created three Powershell scripts: `install.ps1`, `check-connections.ps1`, and `run.ps1`.

Description of `install.ps1`:

- Verifies Python 3.13 and Node are present on the host.
- Creates a Python venv under `backend\venv` and installs `backend\requirements.txt`.
- Installs frontend npm deps and runs the Vite production build.
- Copies the built frontend (`frontend\dist`) into `backend\static`, which FastAPI serves directly (see `StaticFiles` mount in `app\main.py`).

Description of `check-connections.ps1`:

- Preflight check: verifies PostgreSQL, Ollama, and the Arelle taxonomy host are reachable before starting the app.
- Reads connection targets from environment variables already loaded by `run.ps1` (`DB_HOST/DB_PORT`, `OLLAMA_BASE_URL`, `ARELLE_TAXONOMY_URL`).
- Each check is short-timeout and non-fatal by default
- Failures are reported clearly, and the script exit code reflects overall pass/fail so callers can decide whether to abort.

Description of run.ps1:

- Runs the Disclosure Pilot AI reporting app: one FastAPI/uvicorn process serving the API and the built React app (see install.ps1 for the build step).
- Port to listen on. Defaults to 8000.
- Run from the repository root: `.\run.ps1`
- Requires a .env file in the repo root (copy and edit from .env.example).

Gemni Prompt to explain scripts

Assume the user is not familiar with Windows Powershell. In a docx file, explain step by step what each script does, and explain each powershell command once.

Script Explanations

The following explanation of the Powershell scripts was generated by Gemini, based on the prompt above and given the 3 script files.

PowerShell Executive Overview & Beginner Primer

Welcome to the PowerShell automation documentation for the Disclosure Pilot reporting application. If you have never used Windows PowerShell before, command-line scripts can initially look like a complex foreign language. However, PowerShell is simply a tool built into Windows that automates repetitive computer tasks—such as checking network connections, installing software components, and starting applications—by giving direct instructions to the operating system. It prepares the system environment, builds compiled assets, and stages files so the application runs without needing container software like Docker or Podman.

The Disclosure Pilot management suite relies on three core automation scripts written in PowerShell: `check-connections.ps1` (verifies system health), `install.ps1` (builds and installs the application), and `run.ps1` (configures and launches the application). This guide breaks down each script step-by-step so you understand exactly what happens behind the scenes.

 **POWERSHELL SYNTAX QUICK PRIMER:** PowerShell uses small commands called 'Cmdlets' (pronounced command-lets) formatted as Verb-Noun (e.g., `Get-Command`, `Start-Process`). Variables start with a dollar sign (`$variable`) and store data. Symbols like `&` or `|` act as tools to run commands or pass data between them.

Connectivity Preflight Checker (check-connections.ps1)

Before starting a complex application like Disclosure Pilot, it is essential to confirm that all required external services are active and reachable. The check-connections.ps1 script acts as a 'preflight inspector'. It verifies three key dependencies:

PostgreSQL Database: Ensures the main database server is accepting connections on its TCP port.

Ollama AI Engine: Checks that the local Artificial Intelligence service responds to web HTTP requests at its API endpoint.

Arelle XBRL Taxonomy Host: Confirms that financial taxonomy definitions (like US-GAAP 2026 schema standards) can be fetched over the web.

Step-by-Step Execution Workflow

- **Step 1: Set Timeout Parameter:** The script accepts a customizable timeout (defaulting to 4 seconds) so network checks respond quickly without freezing.
- **Step 2: Initialize Results Storage:** An empty results array (\$results) is created to collect the outcome of each connectivity test.
- **Step 3: Define Network Testing Functions:** Two helper functions are declared: Test-TcpPort (opens a raw network connection to check databases) and Test-HttpEndpoint (sends a web request to check AI and taxonomy web hosts).
- **Step 4: Execute Service Connectivity Tests:** The script reads target addresses from environment settings and runs tests against PostgreSQL, Ollama, and Arelle host URLs.
- **Step 5: Output Styled Results & Exit Status:** It prints colored [OK] or [FAIL] lines in the console. If any check fails, it exits with error code 1; if all pass, it exits with code 0.

Technical Deep-Dive & Code Breakdown

The script uses .NET networking objects directly from Windows to test network ports without requiring third-party software :

```
function Test-TcpPort($Name, $HostName, $Port, $TimeoutSec) {  
    $client = New-Object System.Net.Sockets.TcpClient  
    $async = $client.BeginConnect($HostName, [int]$Port, $null, $null)  
    $ok = $async.AsyncWaitHandle.WaitOne($TimeoutSec * 1000)  
    ...  
}
```

In this function, New-Object creates a low-level network client (\$client). It attempts to connect asynchronously to the specified host and port. If the connection succeeds within the timeout

period, it returns a custom object marked `Ok = $true`. For web services, `Invoke-WebRequest` sends a fast 'HEAD' request (or falls back to 'GET') to verify that the HTTP web server responds properly.

PowerShell Command Reference (check-connections.ps1)

Below is an explanation of each PowerShell command and construct introduced in Script 1:

PowerShell Command / Construct	Type	Plain-English Purpose & Action
<code>param(...)</code>	Keyword	Defines input parameters for the script (e.g., <code>TimeoutSeconds = 4</code>).
<code>[int]</code>	Type Cast	Restricts a variable or parameter to strictly hold whole integer numbers.
<code>\$results = @()</code>	Array Syntax	Initializes an empty array structure to store multiple test result items.
<code>function Name()</code>	Keyword	Defines a reusable block of code that carries out a named task.
<code>if / else</code>	Control Flow	Evaluates boolean conditions to execute different blocks of code.
<code>return</code>	Keyword	Exits a function immediately and passes back a result object to the caller.
<code>[PSCustomObject]@{...}</code>	Object Constructor	Creates a structured data object containing customized properties (Name, Ok, Detail).
<code>try { } catch { }</code>	Error Handling	Catches runtime errors gracefully (e.g. network timeout) without crashing.
<code>New-Object</code>	Cmdlet	Instantiates .NET framework objects (e.g., <code>System.Net.Sockets.TcpClient</code>).
<code>[System.Environment]::GetEnvironmentVariable()</code>	.NET Method	Reads environment variables (like <code>DB_HOST</code> or <code>OLLAMA_BASE_URL</code>) set in Windows.
<code>Invoke-WebRequest</code>	Cmdlet	Sends HTTP requests (HEAD / GET) to web services and checks response status.

<code>Write-Host</code>	Cmdlet	Outputs text messages directly into the console window.
<code>-ForegroundColor</code>	Parameter	Sets display color of text printed by Write-Host (e.g., Green, Yellow, Cyan).
<code>foreach (\$item in \$list)</code>	Loop	Iterates through each element in a collection to print individual test results.
<code>-f (Format Operator)</code>	Operator	Fills in placeholders ({0}, {1}) in formatted text strings.
<code>exit <code ></code>	Keyword	Terminates the script process (0 = success, 1 = failure).
<code>\$_</code>	Automatic Var	Represents the current object in pipeline or exception object in a catch block.

Installer & Build Script (install.ps1)

The `install.ps1` script handles the automated setup of both the Python backend and the React frontend of the Disclosure Pilot application. It prepares the system environment, builds compiled assets, and stages files so the application runs smoothly.

Step-by-Step Execution Workflow

- **Step 1: Strict Error Policy:** Sets `$ErrorActionPreference = 'Stop'` so any failure immediately halts execution to prevent half-installed states.
- **Step 2: Prerequisite Validation:** Uses `Assert-Command` (wrapping `Get-Command`) to verify that Python 3.13, Node.js, and npm are installed and available on system PATH.
- **Step 3: Environment Cleanup:** Clears stale `PYTHONHOME` or `PYTHONPATH` variables that could cause Python virtual environment creation to fail.
- **Step 4: Python Virtual Environment Setup:** Creates an isolated Python `venv` at `backend\.venv`, upgrades `pip`, and installs required packages from `requirements.txt`.
- **Step 5: Bytecode Compilation:** Executes Python's `compileall` module on backend source code to catch any syntax errors prior to runtime.
- **Step 6: Frontend Build Execution:** Navigates into frontend directory using `Push-Location`, executes `npm install` and `npm run build`, and restores original directory via `Pop-Location`.

- **Step 7: Static Asset Staging:** Cleans out backend\static, creates a fresh folder, and copies the compiled production frontend dist files into it for FastAPI to serve.

📌 **DIRECTORY STACKING CONCEPT:** Push-Location and Pop-Location act like a directory bookmark system. Push-Location saves your current folder path before entering a subfolder (like frontend), and Pop-Location automatically brings you right back when work is completed.

PowerShell Command Reference (install.ps1)

Below is an explanation of each new PowerShell command and construct introduced in Script 2:

PowerShell Command / Construct	Type	Plain-English Purpose & Action
<code>\$ErrorActionPreference</code>	Preference Var	Configures how PowerShell treats non-terminating errors ('Stop' turns them into terminating exceptions).
<code>\$PSScriptRoot</code>	Automatic Var	Stores the directory path where the currently executing script resides on disk.
<code>Join-Path</code>	Cmdlet	Combines path segments (e.g. root + 'backend') into a valid filesystem path.
<code>Get-Command</code>	Cmdlet	Searches system PATH to verify if an executable (py, node, npm) exists.
<code>throw</code>	Keyword	Generates a custom error exception and halts script execution immediately.
<code>& (Call Operator)</code>	Operator	Executes external commands or programs (e.g. <code>& py -3.13 --version</code>).
<code>2>\$null</code>	Redirection	Suppresses error output streams by sending error messages to null.
<code>[System.Environment]::SetEnvironmentVariable()</code>	.NET Method	Modifies or clears process environment variables (used to clear stale Python variables).
<code>Test-Path</code>	Cmdlet	Checks whether a specified folder or file exists on disk, returning true or false.

\$LASTEXITCODE	Automatic Var	Contains the exit code returned by the last run external program (0 = success).
Push-Location	Cmdlet	Saves current working directory to stack and switches into a target directory.
finally	Block Keyword	Guarantees execution of cleanup code (Pop-Location) even if errors occur inside try block.
Pop-Location	Cmdlet	Restores active working directory back to the folder stored on location stack.
Remove-Item	Cmdlet	Deletes existing folders or files (-Recurse removes subfolders, -Force ignores prompts).
New-Item	Cmdlet	Creates new file system items (-ItemType Directory creates new folders).
Out-Null	Cmdlet	Discards command output so nothing displays on the terminal screen.
Copy-Item	Cmdlet	Copies files/folders from source to destination (-Recurse includes all subfiles).

Application Launcher Script (run.ps1)

The run.ps1 script serves as the production startup command for Disclosure Pilot. It reads configuration from a.env environment file, resolves variable references, configures database connection strings, runs connectivity checks, optionally launches the user's browser, and starts the FastAPI web server.

Step-by-Step Execution Workflow

- **Step 1: Parameter & Switch Parsing:** Accepts optional parameters: -Port (default 8000), -NoBrowser (skip auto-opening browser), and -StrictPreflight (abort on preflight failure).
- **Step 2: File Existence Validation:** Ensures that required environment files (.env) and Python virtual environment executables exist before proceeding.

- **Step 3: Variable Interpolation Engine:** Reads `env` line-by-line using `Get-Content`, parses key-value pairs into a hashtable, and runs up to 10 regex passes to expand nested variable references (e.g. resolving `${WIN_APPIO_DIR}` inside other paths).
- **Step 4: Default Fallbacks & Path Assembly:** Applies default settings for database and AI hosts if omitted, and configures `FRONTEND_DIST` to point directly at `backend/static`.
- **Step 5: Preflight Integration:** Invokes `check-connections.ps1` to test connectivity. If checks fail and `-StrictPreflight` is enabled, startup aborts.
- **Step 6: Background Browser Auto-Launch:** Unless `-NoBrowser` is passed, spawns a background job via `Start-Job` that waits 2 seconds before launching the web browser to target URL.
- **Step 7: Web Server Launch:** Changes folder into `backend` and runs Uvicorn server via Python virtual environment on the designated port.

Deep-Dive: Multi-Pass Variable Expansion & Background Jobs

In configuration files, settings often reference other settings (e.g., `DATA_PATH = '${APP_DIR}\data'`). Standard script readers often fail to expand these, treating `'${APP_DIR}'` literally. The `run.ps1` script solves this by using a loop combined with Regular Expressions (.NET Regex):

```
$maxPasses = 10
for ($pass = 0; $pass -lt $maxPasses; $pass++) {
    $anyExpanded = $false
    foreach ($key in @($envTable.Keys)) {
        $value = $envTable[$key]
        $expanded = [System.Text.RegularExpressions.Regex]::Replace(
            $value, '\$\{([A-Za-z_][A-Za-z0-9_]*)\}',
            { param($m) $refName = $m.Groups[1].Value... }
        )
    }
}
```

Additionally, to launch the browser without blocking the web server startup, `run.ps1` uses `Start-Job`. `Start-Job` creates an independent background process that waits 2 seconds (`Start-Sleep -Seconds 2`) and opens the browser (`Start-Process`) while the main script starts Uvicorn immediately.

PowerShell Command Reference (`run.ps1`)

Below is an explanation of each new PowerShell command and construct introduced in Script 3:

PowerShell Command / Construct	Type	Plain-English Purpose & Action
<code>[switch]</code>	Parameter Type	Defines a boolean toggle flag on command line (true if specified, false if omitted).
<code>@{}</code>	Hashtable Syntax	Initializes a key-value pair dictionary structure (\$envTable).
<code>Get-Content</code>	Cmdlet	Reads the contents of a text file (.env) line-by-line into PowerShell.
<code>ForEach-Object</code>	Cmdlet	Processes each item coming through the pipeline individually.
<code>-split</code>	Operator	Splits strings into an array based on a delimiter character (e.g. splitting key=value).
<code>.Trim() /.StartsWith()</code>	.NET Methods	Trims leading/trailing spaces or tests string prefix during file parsing.
<code>for (\$i=0; \$i -lt \$max; \$i++)</code>	Loop	Standard counter loop for repeating code blocks a fixed number of times.
<code>[Regex]::Replace()</code>	.NET Method	Replaces regular expression pattern matches with dynamic values (used for .env expansion).
<code>Start-Job</code>	Cmdlet	Runs a block of code asynchronously as a background process thread.
<code>Start-Sleep</code>	Cmdlet	Pauses execution for a specified duration in seconds (-Seconds 2).
<code>Start-Process</code>	Cmdlet	Launches an executable application or opens a URL in default web browser.
<code>\$using:Port</code>	Scope Modifier	Allows background jobs (Start-Job) to access variables defined in script scope.

Script Blocking Issue

To make PowerShell fully accept and output Unicode (UTF-8), enter:

```
$OutputEncoding = [Console]::InputEncoding = [Console]::OutputEncoding =
[System.Text.Encoding]::UTF8
```

Set your user account permanently (doesn't need admin) to

RemoteSigned (rather than Bypass) as the safer permanent choice — it

allows locally-created scripts (like these) to run freely, but still

requires a digital signature for scripts downloaded from the internet,


```
# so you're not fully disabling the protection, just exempting your own
# local files. Enter:
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

```
# Since install.ps1 was downloaded through the browser/chat interface,
# Windows tagged it with a hidden "Mark of the Web" (an NTFS alternate
# data stream called Zone.Identifier), so PowerShell treats it as remote
# — hence the identical error even after changing the policy.
```

```
# Enter directly:
Unlock-File .\unlock.ps1
# Then call it to unblock the scripts below
.\unlock.ps1
```

```
# Fix — strip that tag from each file:
Unlock-File .\install.ps1
Unlock-File .\run.ps1
Unlock-File .\check-connections.ps1
Unlock-File .\test-backend.ps1
Unlock-File .\test-frontend.ps1
Unlock-File .\backend\Activate.ps1
Unlock-File .\backend\Deactivate.ps1
```

Section 11 - Code Development Prompting

This section outlines best practices for prompting Large Language Models (LLMs) to generate a realistic backend (Python). The following methodology was successfully employed using **Claude Sonnet 4.6/5** to build a comprehensive internal financial reporting application.

1. Top-Level Instructions (Instructions.docx)

The following high-level prompt establishes the global context, technology stack, and architectural boundaries for the LLM.

System Prompt

OBJECTIVE:

1. Create an internal Windows application for generating quarterly (10-Q) and yearly (10-K) financial reports.

2. Adhere strictly to the following technology stack:

Layer	Technology
OS	Windows 11
Frontend	React 18, Vite, Recharts
Backend	FastAPI, Uvicorn, Python 3.11
ORM	SQLAlchemy 2.0
Validation	Pydantic v2
Authentication	JWT
Database	PostgreSQL 18
AI Workflow	LangGraph, LangChain
LLM	ChatOllama (gemma3)
Docx Generation	python-docx, docxcompose

3. DATABASE READ ACCESS: The core financial database is already populated. The application must treat financial tables as READ-ONLY. Refer to `gem-db-init.sql` for schema structures.

4. DATABASE WRITE ACCESS: User authentication details must be stored and managed in a separate database schema. Refer to `app-users-spec.txt`.

5. ENVIRONMENT: Refer to `.env.example` and `yml-notes.txt` for environment configuration variables.

6. BACKEND SPECIFICATION: Implement backend business logic as detailed in `Backend-Design.docx`.
7. FRONTEND SPECIFICATION: Implement frontend user interfaces as detailed in `Frontend-Design.docx`.

Engineering Insights & Analysis

- Context Setting (Obj. 1): Instantly anchors the LLM to the domain-specific nuances of SEC financial reporting (10-Q and 10-K), preventing generic code generation.
- Stack Constraints (Obj. 2): Because desktop and enterprise applications can be engineered in countless configurations, explicitly locking down libraries prevents the AI from mixing incompatible paradigms.
- State Management & Permissions (Obj. 3 & 4): Explicitly partitioning read-only data (financials) from read/write data (user credentials) avoids architectural confusion and protects production data.
- Infrastructure as Code (Obj. 5, 6, & 7): Modern LLMs excel at infrastructure configuration. Providing environment templates ensures the generated Containerfile and compose files sync perfectly with the code.
- Iterative Decomposition (Obj. 8 & 9): AI possesses vast baseline knowledge of web architecture. To get predictable results, you must restrict its focus to the specific UI layouts and endpoints defined in your sub-specifications. Expect multiple iterative refinements to perfect your layouts and logic.

2. Backend Specifications (Backend-design.docx)

This prompt narrows the scope to specific implementation patterns, security protocols, and data layers.

Detailed Backend Prompt

INSTRUCTIONS:

1. Architecture: Implement code using clean Python class structures.
2. Data Access: Use SQLAlchemy 2.0 to interface with the PostgreSQL database (refer to `gem-db-init.sql`).
3. Query Pattern: Use the modern `self.db.execute(stmt).scalar()` syntax for queries. Do NOT use the legacy `self.db.query()` syntax.
4. User Security Schema: Refer to `app-users-spec.txt` for handling application user accounts.

5. Authentication Handler: Use the ``fastapi.security`` module to manage backend auth. The backend must explicitly handle:
 - User account registration
 - Secure password resets
 - Standard user login
6. Session Management: Implement stateful JWT tokens. Initiate the application workflow by serving the login form to the frontend client.
7. Post-Auth Routing: Upon successful JWT validation, serve ``index.html`` to the frontend client.
8. Data Validation: Enforce strict request and response validation using Pydantic v2 schemas.
9. Hardware Acceleration: Integrate the NVIDIA Container Toolkit to leverage GPU CUDA processing capabilities.
10. Reporting Module A: Refer to ``Form-10-Q-design.docx`` for granular implementation logic.
11. Reporting Module B: Refer to ``Form-10-K-design.docx`` for granular implementation logic.
12. Testing: Generate comprehensive test suites using ``pytest`` with an in-memory ``sqlite`` database layer.

Engineering Insights & Analysis

- Coding Style (Obj. 1): Without specifying object-oriented class structures, LLMs defaults to scripting or functional paradigms arbitrarily.
- Enforcing Modern Syntax (Obj. 3): LLMs are often trained on outdated documentation and legacy codebases. Explicitly banning patterns like `db.query()` forces compliance with modern SQLAlchemy 2.0 standards.
- Handling Context Ambiguity (Obj. 4 & 5): Clarifying that "users" refers to *application accounts* rather than *database connection users* prevents major security holes. Outlining edge cases like "password resets" ensures the AI covers basic features it might otherwise skip.
- Lifecycle & Tooling (Obj. 6, 7, 8 & 9): Establishes explicit execution order and hardware dependencies up front.
- Modularity (Obj. 10, 11): Refers to lower level details separately to hint at a modular design.
- Test Isolation (Obj. 12): Asking the AI to mock PostgreSQL with SQLite in pytest ensures fast, reliable unit testing without risking live data environments.

3. Financial Reporting Specifications (Form-10-K-design.docx)

As the system moves toward core financial data generation, the level of instructional detail increases significantly.

Detailed 10-K Prompt Excerpt

INSTRUCTIONS:

Upon receiving a frontend request for a 10-K filing compilation, compute and structure the following data:

ITEM 8: Financial Statements and Supplementary Data

Query the database general ledger and chart of accounts (see 'gem-db-init.sql') to dynamically compute and construct the following outputs:

- A detailed Balance Sheet spanning two (2) trailing years.
- An Income Statement spanning three (3) trailing years.
- A Cash Flow Statement spanning three (3) trailing years.

Engineering Insights & Analysis

- **Dynamic Synthesis vs. Static Views:** LLMs struggle when required to aggregate multi-year data dynamically from a raw transactional ledger rather than pulling from pre-formatted database tables. The code must map disparate accounts into a unified, chronologically sequenced layout.
- **The "Excel Reference" Strategy:** Claude frequently struggles with the abstraction of complex, multi-year ledger aggregations. To resolve this, feed the LLM actual mock spreadsheet structures (e.g., bal_format.xlsx). Providing explicit backend schemas and target frontend layouts provides the exact blueprint the AI needs to generate flawless parsing logic.

Summary Principles for AI Software Engineering

1. Top-Down Architectural Decomposition: Always structure your development prompts from macroscopic system requirements down to microscopic, procedural logic.

2. Modular File-Based Contexts: Group prompts into specialized specification files that mirror your intended software directory layout. This prevents context pollution and keeps code generations clean and maintainable.

Reference Materials:

Full-Stack AI Development Insights:

https://dev.to/mr_nova/using-chatgpt-and-claude-for-full-stack-app-development-cgm

Building Production Apps with LLMs:

<https://leadershiplighthouse.substack.com/p/how-i-built-a-production-app-with>

Rapid Full-Stack Engineering Case Study:

<https://shiftmag.dev/how-i-built-a-full-stack-app-in-6-days-with-the-help-of-ai-7564>

Section 12 - iXBRL Prompting

This section details an iterative prompting strategy for engineering a Python-based backend file convertor using Large Language Models (LLMs). This exact methodology was successfully leveraged with **Claude Sonnet 4.6/5** to develop an **iXBRL** transformer capable of generating EDGAR-compliant 10-K and 10-Q filings in HTML format. The **Arelle** iXBRL validator was used to verify the filings.

1. What is iXBRL?

To understand the complexity of the EDGAR iXBRL filing format, a number of terms must be defined.

XBRL (eXtensible Business Reporting Language) is the digital barcoding system for business data. Instead of presenting financial statements as flat text or static PDFs, XBRL assigns a unique, standardized **element**(or **concept**) to every single metric (such as "Revenue" or "Net Income") based on an official **taxonomy**. This transforms financial reports into machine-readable data, allowing regulators, investors, and automated software to instantly extract, compare, and analyze corporate performance without the headache of manual data entry.

iXBRL (inline XBRL) takes this efficiency and gives it a human-friendly face. In the early days of XBRL, companies had to prepare two separate files: a formatted document for humans to read and a clunky XML file for computers to scan. iXBRL solves this double-work by embedding the XBRL **elements** directly into a standard HTML web page. The result is a single, unified document that renders as a beautifully formatted, readable financial report on any web browser, while automated software can simultaneously peer under the hood to extract the exact data points.

Taxonomy is a comprehensive, structured digital dictionary of financial and non-financial reporting terms. Most US filers use the standard **US GAAP Financial Reporting Taxonomy**. A taxonomy is much more than a flat list of terms. It is composed of two primary layers that work together, **elements** and **linkbases**.

Example **element** for Total Revenues on Income Statement:

```
<!-- Core element for Revenues -->  
<xs:element name="Revenues"  
            id="us-gaap_Revenues"
```

```
type="xbrli:monetaryItemType"
substitutionGroup="xbrli:item"
xbrli:periodType="duration"
xbrli:balance="credit"/>
```

Example **Label Linkbase** which connects the element tag to human-readable text labels:

<!-- STEP 1: Locate the raw element in the core taxonomy schema -->

```
<loc xlink:type="locator"
  xlink:href="us-gaap-2026.xsd#us-gaap_Revenues"
  xlink:label="loc_Revenues"/>
```

<!-- Total Label (Used at the bottom of an Income Statement section) -->

```
<label xlink:type="resource"
  xlink:role="http://www.xbrl.org/2003/role/totalLabel"
  xlink:label="lab_Revenues_total"
  xml:lang="en-US">Total Revenue</label>
```

<!-- STEP 3: Tie the locator to the labels using Arcs -->

```
<labelArc xlink:type="arc"
  xlink:arcrole="http://www.xbrl.org/2003/arcrole/concept-label"
  xlink:from="loc_Revenues"
  xlink:to="lab_Revenues_total"/>
```

The htm file already has the proper label: “Total Revenues”. **Why do we need a label linkbase?**

When automated software or the SEC processes a 10-K filing, they don't look at the pretty HTML page. They use an XBRL parser to pull the **raw facts** out into a database. **Once extracted**, the **HTML layout disappears**. The software is left with a flat database record that looks like this:

- **Concept:** us-gaap:Revenues
- **Value:** 15500000
- **Context:** FY2025

If an automated software program (like Bloomberg) wants to display this data inside their own custom data grid without re-rendering the company's original, messy HTML website, it queries the **Label Linkbase** to find out what that tag is officially called in plain English, e.g. “Total Revenue”

However, a single element can mean different things depending on where it appears in a 10-K. The label linkbase allows the same exact tag to change its display name. Since the linkbase is not stored along with the element, how can the automated software know what display label to use?

Since the HTML file, the taxonomy file, and the label linkbase file are all physically separate documents, how does the analyst software bridge the gap? The missing piece of the puzzle is another linkbase file: the Presentation Linkbase.

The **Presentation Linkbase** acts as the "director" or layout map for the report. It tells the software exactly how to arrange the data on a screen and explicitly dictates **which label role** to fetch from the label dictionary for **any given section**, such as **Income Statement**.

Here is exactly how the analyst software connects the dots across these separate files. the software resolves what label to display by following a strict three-step chain of command:

1. Raw facts from 10-K HTML → Look up position → 2. Presentation Linkbase
Finds "preferredLabel" → 3. Label Linkbase → Extract Text → Human Sees: "Total Revenue"

The analyst software checks the Presentation Linkbase to see where us-gaap:Revenues sits in the report structure. The preferredLabel attribute explicitly tells the software: "When displaying this specific tag inside the **Income Statement** section, do not use the **default label**. Use the **totalLabel** role instead."

XML

```
<presentationArc xlink:type="arc"
    xlink:arcrole="http://www.xbrl.org/2003/arcrole/parent-child"
    xlink:from="loc_IncomeStatementLineItems"
    xlink:to="loc_Revenues"
    order="5.0"
    preferredLabel="http://www.xbrl.org/2003/role/totalLabel"/>
```

A Simple iXBRL HTML Example

Here is a short, simplified example of how Inline XBRL tags (using the <ix:> namespace) wrap around standard HTML text to make a financial figure readable by both humans and machines:

HTML

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:ix="http://www.xbrl.org/2013/inlineXBRL"
      xmlns:us-gaap="http://fasb.org/us-gaap/2026"
      xmlns:iso4217="http://www.xbrl.org/2003/iso4217">
<head>
  <title>Inline XBRL Demonstration</title>
</head>
<body>
  <h2>Financial Highlights</h2>
  <p>
    For the fiscal year ended December 31, 2025, the Company is pleased to report that its
    <strong>
      <ix:nonNumeric name="us-gaap:Revenues " contextRef="FY2025">Total
Revenue</ix:nonNumeric>
    </strong>
    was
    $<ix:nonFraction name="us-gaap:Revenues "
      contextRef="FY2025"
      unitRef="usd"
      decimals="0"
      scale="0">15,500,000</ix:nonFraction>.
  </p>
</body>
</html>

```

What's happening here: A human viewing this in a browser simply sees: *"its Total Revenue was \$15,500,000."* However, an XBRL parser reads the `<ix:nonNumeric>` and `<ix:nonFraction>` tags and instantly logs that the GAAP accounting concept 'Total Revenue' for the 'FY2025' context equals 15500000 US Dollars.

Where are the `xs:element name="Revenues"` and label `xlink: Total Revenue`? They are hidden in configuration files, listed below, which are separate from the html file.

It looks exactly like a normal HTML webpage or document but contains **hidden iXBRL tags** directly behind the financial text. The iXBRL format allows analysts and regulators to read the

financial statement as a standard HTML document while allowing automated software to read the data-rich tags in the background. The **SEC mandates** that this **structured format** cover all **financial statements, footnotes**, and the **cover page** information. Here, footnotes means “**Notes to Consolidated Financial Statements**”, which is contained in item 8 of the Form 10-K, and item 1 of the Form 10-Q. In addition to the main HTML file (with extension.htm), some iXBRL configuration files must be developed, as listed in the following table, with the file suffixes: .xsd, _pre.xml, _cal.xml, _def.xml, and _lab.xml.

Summary Checklist for a 10-K Submission Bundle

File Extension	Component Type	Purpose	Required?
.htm	Inline XBRL Document	The actual HTML report containing embedded financial tags.	Yes
.xsd	Extension Schema	Declares custom elements and metadata blueprints.	Yes
_pre.xml	Presentation Linkbase	Defines the organizational hierarchy and display order.	Yes
_cal.xml	Calculation Linkbase	Defines mathematical relationships (e.g., \$A + B = C\$) for validation.	Yes
_def.xml	Definition Linkbase	Establishes dimensional relationships for complex tables/footnotes.	Yes
_lab.xml	Label Linkbase	Attaches human-readable text labels to standard and custom elements.	Yes

2. Top-Level Instructions (Instructions.docx)

The following high-level prompt to Claude establishes the global context:

For the 10-Q and 10-K , the SEC requires xbrl xhtml format. Research how we can create a file transformer functon to add xbrl tagging to the 10-Q and 10-K html files.

First, we need us-gaap mappings like this:

"accounts receivable":"us-gaap:AccountsReceivableNetCurrent".

Second, we need to parse the html and add the special xbrl tags.

3. First Iteration

Claude created the mappings, like this:

Label	iXBRL Tag
"cash and equivalents":	{"concept": "us-gaap:CashAndCashEquivalentsAtCarryingValue", "type": "instant", "balance": "debit", "negate": False}
"short-term investments":	{"concept": "us-gaap:ShortTermInvestments", "type": "instant", "balance": "debit", "negate": False}
"accounts receivable":	{"concept": "us-gaap:AccountsReceivableNetCurrent", "type": "instant", "balance": "debit", "negate": False}
"inventory":	{"concept": "us-gaap:InventoryNet", "type": "instant", "balance": "debit", "negate": False}
"property, plant & equip":	{"concept": "us-gaap:PropertyPlantAndEquipmentGross", "type": "instant", "balance": "debit", "negate": False}
"total assets":	{"concept": "us-gaap:Assets", "type": "instant", "balance": "debit", "negate": False}
...	...

This is just a small sample of the complete list of iXBRL mappings. With just one example in the prompt instructions, Claude was able to lookup the whole mapping list based on the labels used in the financial tables for this application. Then Claude created the python data structure to hold the mappings. All done within a few minutes. However, the actual iXBRL tagging code took several iterations to complete without errors.

4. Error Analysis (arelle-msg-log-1.txt)

The **Arelle** xbrl file validator was used to check the 10-K htm file after each iteration. Then the message log, such as arelle-msg-log-1.txt, was uploaded to Claude to analyze and fix any issues in the iXBRL transformer.

A few examples from **Arelle validation**:

The context identifier CIK 1234567 is not 10 digits - sec-10-k.xhtml 29

Warning, Inline XBRL ix:header is recommended to be nested in a <div> with style display:none

Instance fact missing schema definition: us-gaap:CostOfGoodsSold of Inline Element ix:nonFraction - sec-10-k.xhtml 514

Inline element ix:nonFraction fact **us-gaap:IncomeTaxExpenseBenefit** fact us-gaap:IncomeTaxExpenseBenefit type decimal transform ixt:num-dot-decimal **value error: -7009009** - sec-10-k.xhtml 574

Example of **Claude fix**:

For the value error: -7009009

Comment: Negative XBRL values must be expressed using the sign="-" attribute.

HTML update:

```
<td class="number"><ix:nonFraction name="us-gaap:IncomeTaxExpenseBenefit" sign="-"
contextRef="ctx_dur_20250101_20251231" unitRef="USD" decimals="0" id="f4de28396"
format="ixt:num-dot-decimal">7009009</ix:nonFraction></td>
```

5. Final Version (xbrl_tagger.py)

Besides the python code, **xbrl_tagger.py**, the following configuration files were developed for handling iXBRL:

tifx-20251231.xsd

tifx-20251231_cal.xml

tifx-20251231_def.xml

tifx-20251231_lab.xml

tifx-20251231_pre.xml

Note that _lab.xml includes only a few custom labels, because most data uses standard labels.

For this demonstration application, the backend codebase is pre-configured with a suite of the most common financial notes (currently just 8, but designed to be extended). Users select the specific notes required for a given filing cycle (e.g., selecting 6 notes in Year 1 and 8 notes in Year 2).

Each note requires precise iXBRL tags mapped to its specific contents; for example, the Income Taxes note relies on the us-gaap:IncomeTaxExpenseBenefit tag. If this mapping is not hardcoded, programmatically distinguishing between different financial values (such as a tax expense versus pre-tax income, which are both raw dollar amounts) becomes incredibly complex.

Because the Notes section was generated by ChatOllama LLM, Claude evaluated numerous scenarios to tag each note accurately. A key breakthrough was adopting a hybrid approach: combining ChatOllama's text narrative with deterministic financial data to eliminate hallucinated numbers. Even then, narratives were sometimes unreliable. It was the Gemini LLM that advised a refactoring of Python code using a more advanced ollama model, looping over each note separately to avoid token limits, and setting enhanced ChatOllama arguments.

After many many iterations, all issues were fixed, and the 10-Q and 10-K files were successfully converted to iXBRL format, acceptable by the SEC EDGAR system, according to the **Arelle** validator.

XBRL References:

<https://www.xbrl.org/the-standard/what/ixbrl/>

<https://www.xbrl.org/the-standard/how/getting-started-for-developers/>

<https://www.xbrl.org/the-standard/why/ai-future/>

<https://www.sec.gov/data-research/structured-data/inline-xbrl>

<https://www.sec.gov/files/edgar/filer-information/specifications/xbrl-guide.pdf>

<https://specifications.xbrl.org/work-product-index-group-base-spec-base-spec.html>

<https://www.sec.gov/search-filings>

Arelle References:

<https://arelle.readthedocs.io/en/2.41.6/>

<https://arelle.org/arelle/pub/>

Section 13: Testing

Testing financial applications comes with its own set of challenges. While the previous section focused on validating iXBRL filing files, this section covers traditional unit and integration testing. To ensure complete data integrity, you need to verify several critical stages of the data pipeline:

- **Data Extraction:** Confirming that the program extracts the precise data required from the database.
- **Data Transformation:** Verifying that data is correctly transformed from the general ledger into financial statements, such as the balance sheet.
- **Data Transmission:** Ensuring that information transfers seamlessly and accurately from the backend to the frontend.
- **Data Presentation:** Validating that the final user interface correctly displays the intended data.

Testing Methodologies

To address these complexities, a combination of automated constraints, independent cross-validation, and specialized testing frameworks was implemented.

1. Built-In Validation

Certain fundamental accounting principles serve as built-in tests within the application. For example, the balance sheet uses the core accounting equation to ensure that total assets always equal the sum of total liabilities and total equity.

2. Independent Cross-Validation (LLM Auditing)

To rigorously confirm that the primary application (built by Claude) transformed data into the financial statements correctly, a separate, independent program was developed using a different LLM: Meta AI.

- This secondary program independently performed the exact same data transformations.
- The outputs—including balance sheets, income statements, and cash flow statements—were then systematically compared against each other to identify and resolve any discrepancies.

3. Framework-Specific Testing

Comprehensive unit testing was integrated across both layers of the application's architecture:

- **Backend:** Unit level tests were executed using the pytest framework.
- **Frontend:** User interface and component functionality were validated using Vitest and React Testing Library.

Backend Tests (test_backend.py)

Uses pytest with a SQLite in-memory database. All tests run without a live PostgreSQL instance — schema-qualified table names (e.g. bookkeeper.general_ledger) are transparently rewritten to flat SQLite equivalents by a session-level fixture that patches session.execute.

Test QuarterDates

Tests the `_quarter_dates` and `_year_dates` helper functions that compute fiscal period boundaries.

Test	What it checks
test_q1	Q1 runs from January 1 to March 31.
test_q2	Q2 runs from April 1 to June 30.
test_q3	Q3 runs from July 1 to September 30.
test_leap_year_q1	Q1 end date is still March 31 in a leap year (2024).
test_year_dates	Annual period runs from January 1 to December 31.

Test GetChartOfAccounts

Tests the `get_chart_of_accounts` function that fetches account definitions from the database.

Test	What it checks
test_returns_all_accounts	All 17 seeded chart-of-accounts rows are returned.
test_fields_present	Each row contains account, acct_name, category, and acct_subtype keys.

Test	What it checks
test_ordered_by_acct_id	Rows are returned in ascending account number order.
test_category_mapping	Account 1000 (Cash and Equivalents) is correctly categorized as "Asset".
test_revenue_accounts_present	Exactly two revenue accounts exist in the seeded data.

Test GLBalancesCumulative

Tests get_gl_balances_cumulative, which sums debit-minus-credit balances through a given date (used for balance sheet accounts).

Test	What it checks
test_cash_balance_after_q1	Cash account (1000) has a positive cumulative debit balance through 2024 Q1.
test_excludes_retained_earnings	When exclude_acct_type is passed, account 3100 (Retained Earnings) is absent from the result.
test_excludes_dividends	The dividends account (7000) is also excluded when the retained-earnings filter is applied.
test_no_future_entries	2024 Q1 GL entries do not appear in balances computed through 2023-12-31.

Test ComputeRetainedEarnings

Tests _compute_retained_earnings, which calculates cumulative retained earnings from the opening balance through any given date.

Test	What it checks
test_includes_opening_balance	As of 2022-12-31, retained earnings equals the seeded opening balance of \$15,000,000.
test_adds_net_income	After 2023 activity (revenue \$1M, expenses \$500K), retained earnings increases by \$500K.
test_subtracts_dividends	2024 Q1 dividends of \$25K are subtracted from retained earnings.
test_prior_period_less_than_current	Retained earnings at 2024 Q1 end are greater than at 2023 year-end (net income positive).

Test BuildBalanceSheet

Tests build_balance_sheet, which produces a two-period balance sheet from the general ledger.

Test	What it checks
test_returns_list_of_rows	Returns a non-empty list of balance sheet row objects.
test_no_dividends_row	The Dividends Paid account does not appear as a balance sheet line (it is an equity adjustment, not a balance sheet item).
test_no_retained_earnings_double_count	Exactly one Retained Earnings row exists — it is not duplicated by the chart-of-accounts loop.
test_retained_earnings_value	The Retained Earnings row value matches the result of compute_retained_earnings for the same date.
test_has_asset_liability_equity_rows	The output contains rows from all three balance sheet categories.

Test BuildIncomeStatement

Tests `build_income_statement`, which produces a multi-period income statement with optional year-to-date columns.

Test	What it checks
<code>test_returns_revenue_and_expense_rows</code>	Output contains both Revenue and Expense category rows.
<code>test_revenue_positive</code>	All revenue rows have a non-negative <code>current_period</code> value (credit-normal accounts).
<code>test_ytd_columns_populated</code>	When YTD date ranges are provided, <code>ytd_current</code> and <code>ytd_prior</code> are populated on every row.
<code>test_ytd_none_when_not_requested</code>	When no YTD dates are passed, <code>ytd_current</code> is <code>None</code> on all rows.

Test BuildCashFlow

Tests `build_cash_flow`, which produces a classified cash flow statement.

Test	What it checks
<code>test_returns_all_sections</code>	All required section headers and summary lines are present (Operating, Investing, Financing, Net Increase in Cash, Beginning/Ending Cash).
<code>test_net_income_positive</code>	Net Income is positive because seeded revenue exceeds seeded expenses.
<code>test_dividends_negative</code>	Dividends Paid is zero or negative, reflecting a cash outflow.
<code>test_cash_end_equals_begin_plus_net</code>	Cash at End of Period equals Cash at Beginning of Period plus Net Increase in Cash.

Test GetQuarterlyStatements

Tests the `get_quarterly_statements` facade that assembles all three statements for a given quarter.

Test	What it checks
<code>test_returns_seven_values</code>	Returns a 7-tuple: balance sheet, income statement, cash flow, period label, prior label, balance sheet prior label, and dates dict.
<code>test_period_label_format</code>	The current-period label contains the correct quarter-end month and year (e.g. "March 31, 2024").
<code>test_bs_prior_label_is_year_end</code>	The balance sheet prior-period label references the prior December 31 year-end, not a quarter end.
<code>test_prior_label_is_quarterly</code>	The income statement prior-period label references the same quarter in the prior year.
<code>test_dates_dict_keys</code>	The returned dates dictionary contains all seven expected keys.
<code>test_invalid_quarter_raises</code>	Q1, Q2, and Q3 all complete without error (Q4 is reserved for yearly reports and is rejected at the API layer).

Test Report 10Q Endpoints

Tests the FastAPI `/api/10q/financials` endpoint using `TestClient` with dependency overrides for the database session and authentication.

Test	What it checks
<code>test_financials_endpoint_200</code>	A valid POST with year 2024, quarter 1 returns HTTP 200.
<code>test_financials_response_has_balance_sheet</code>	The response body contains a non-empty <code>balance_sheet</code> array.

Test	What it checks
et	
test_financials_response_has_labels	The response body contains period_label, prior_label, and bs_prior_label fields.
test_financials_invalid_quarter_rejected	Posting quarter 4 returns HTTP 400.
test_financials_requires_auth	Without the auth override in place, the endpoint returns 401 or 403.

Test AuthService

Tests user creation, password hashing, and password reset directly against the database.

Test	What it checks
test_password_hash_and_verify	A hashed password verifies correctly; a wrong password does not.
test_create_user_in_db	A new user row can be inserted and retrieved, and the stored hash matches the original password.
test_duplicate_email_rejected	Inserting two rows with the same email raises a database exception (unique constraint).
test_reset_password_updates_hash	After updating password_hash, the new password verifies and the old one does not.

Frontend Tests (test_frontend.test.jsx)

Run with:

Uses Vitest and React Testing Library. All API calls are intercepted via `vi.mock("../api/client")`. React-router-dom and `useAuth` are also mocked so individual components render in isolation without a router or auth context.

BalanceSheetTable

Tests the balance sheet display component in isolation using fixture data.

Test	What it checks
renders the Balance Sheets title	The heading "Balance Sheets" is present in the DOM.
shows current period column header from <code>dates.quarter_end</code>	The column header is derived from <code>dates.quarter_end</code> and displays as "March 31, 2024".
shows prior period column header as Year ended Dec 31	The prior-period column shows the <code>bsPriorLabel</code> prop (year-end label).
renders ASSETS, LIABILITIES, EQUITY section headers	All three section group headers are rendered.
renders account names	Individual account names such as "Cash and Equivalents" and "Retained Earnings" appear in the table.
renders Total Assets row	A "Total Assets" summary row is present.
renders Total Liabilities + Equity row	A "Total Liabilities + Equity" summary row is present.
renders without crashing when data is	Passing an empty data array still renders the title without errors.

Test	What it checks
empty	
falls back to bsPriorLabel when dates.prior_year_end is absent	When the dates object is empty, the component uses the bsPriorLabel prop for the prior-period column header.
shows dashes for zero values	Accounts with zero balances display "—" rather than "\$0".

NavBar

Tests the top navigation bar for the presence of all required links.

Test	What it checks
renders Home link	A "Home" link is in the navbar.
renders Quarterly 10-Q link	A link containing "10-Q" is present.
renders Yearly 10-K link	A link containing "10-K" is present.
renders Help link	A "Help" link is present.

LoginForm

Tests the login/sign-up/reset-password form component.

Test	What it checks
renders email and password fields	Both email and password inputs are present and properly labelled.

Test	What it checks
renders login button	A "Sign In" button is present in the default login mode.
shows toggle to sign-up mode	Clicking the sign-up toggle switches the form to sign-up mode and shows a sign-up submit button.
shows toggle to reset password mode	Clicking the reset-password toggle switches the form and shows a reset/send button.
calls onLogin with token on successful login	On a successful API response, the onLogin callback is called with the JWT access token.
shows error message on failed login	On a 401 response, an error message is displayed.
does not submit with empty fields	Clicking Sign In with blank fields does not make an API call.

Prep10QPage

Tests the quarterly report preparation page.

Test	What it checks
renders year and quarter dropdowns	Both year and quarter selects are present and associated to their labels.
year dropdown contains current and 4 prior years	The year select has exactly 5 options.
quarter dropdown only shows Q1, Q2, Q3	Q1, Q2, Q3 options exist and Q4 does not.
renders Save	A save button is present.

Test	What it checks
Financials button	
renders Save MD&A button	A "Submit MD&A" button is present on initial render.
renders Generate 10-Q Report button	A "Generate 10-Q Report" button is present on initial render.
fetches financials when year/quarter selected	After selecting year 2024 and quarter 1 then clicking "Load Financial Statements", <code>tenQApi.getFinancials</code> is called with (2024, 1).
shows acknowledgement after Save Financials	After loading financials then clicking "Save Financials", a success message referencing <code>item010.docx</code> appears.

Report10QPage

Tests the page for generating the final 10-Q report document.

Test	What it checks
renders year and quarter dropdowns	Both selects are present and properly labelled.
renders Generate Report button	A "Generate 10-Q Report" button is present.
shows acknowledgement after report generation	Clicking the generate button triggers the API call and displays a success message referencing <code>sec-10-q.docx</code> .

Edgar10QPage

Tests the page for converting the 10-Q report to SEC EDGAR HTML format.

Test	What it checks
renders Generate EDGAR HTML button	The generate-HTML button is present.
shows acknowledgement after HTML generation	Clicking the button triggers the API call and displays a success message referencing sec-10-q.htm.

Prep10KPage

Tests the yearly report preparation page.

Test	What it checks
renders year dropdown only (no quarter)	A year select is present, and no quarter select exists (yearly reports do not use quarters).
year dropdown contains current and 4 prior years	The year select has exactly 5 options.
renders Save MD&A button	A "Submit MD&A" button is present on initial render.
renders Generate 10-K Report button	A "Generate 10-K Report" button is present on initial render.

HomePage

Tests the application home page.

Test	What it checks
renders welcome message	The home page displays a message matching "Generate professional quarterly and yearly financial reports".

HelpMenu

Tests the Help dropdown with User Guide and About dialogs.

Test	What it checks
renders User Guide link	A "User Guide" button/link is present.
renders About link	An "About" button/link is present.
User Guide dialog opens on click	Clicking "User Guide" opens a dialog whose body contains application description text.
About dialog shows version number	Clicking "About" opens a dialog displaying a version number.

React Testing References:

<https://fastapi.tiangolo.com/reference/testclient/>

<https://dev.to/felipewind/python-fastapi-tutorial-to-test-http-client-requests-2ah9>

<https://jnikenoueba.medium.com/unit-and-integration-testing-with-fastapi-e30797242cd7>

<https://medium.com/@dilip.bhaidiya/mastering-react-js-testing-with-jest-a-comprehensive-guide-1acada2b9586>

<https://www.browserstack.com/guide/selenium-react-tutorial>

Python Testing References:

<https://realpython.com/pytest-python-testing/>

<https://circleci.com/blog/pytest-python-testing/>

<https://medium.com/@ronakchitlangya1997/pytest-a-beginner-guide-f9d4919cd427>

Section 14: Tools

A variety of software tools were used to build the application. This section provides a brief overview of each tool's functional description and competitive advantage.

Frontend Development & Visualization

React

React is an open-source JavaScript library developed by Meta for building dynamic, component-based user interfaces. It uses a Virtual DOM to optimize rendering performance by only updating components whose data changes. Compared to Angular, React is significantly more lightweight, unopinionated, and features a flatter learning curve. It gives developers total architectural freedom and benefits from a massive ecosystem, making it easier to integrate third-party libraries without forcing a rigid framework structure.

Vite

Vite is a modern frontend build tool that leverages native ES modules in the browser to deliver instantaneous server start times and lightning-fast Hot Module Replacement (HMR) during local development. Compared to Webpack, Vite provides a drastically faster developer experience by avoiding the need to pre-bundle the entire application before spinning up a local environment. Production builds are managed via highly optimized Rollup configurations out of the box.

Recharts

Recharts is a composable charting library built specifically for React applications. It leverages SVG elements under the hood to construct highly customizable, responsive, and reusable data visualization components. Compared to Chart.js (or its React wrappers), Recharts fits natively into the React lifecycle and component paradigm. It allows developers to customize declarative sub-components (like XAxis, Tooltip, or Bar) as direct children, rather than forcing configuration through large, complex JavaScript objects.

Backend & Core Python Services

FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python, fully powered by standard Python type hints. It natively supports asynchronous programming out-of-the-box using the ASGI specification. Compared to Flask, FastAPI delivers vastly superior performance metrics comparable to Node.js and Go, alongside automated interactive

documentation generation (via Swagger UI). It eliminates extensive boilerplate code by using native asynchronous execution and strict type checking.

Uvicorn

Uvicorn is a lightning-fast ASGI (Asynchronous Server Gateway Interface) web server implementation for Python. It acts as the production-grade engine running asynchronous frameworks like FastAPI or Starlette. Compared to Gunicorn (traditionally used for WSGI applications), Uvicorn is built specifically to process non-blocking asynchronous connections natively using `uvloop`. It allows a single process to handle thousands of concurrent requests smoothly without crashing or stalling.

Python

Python is a high-level, interpreted programming language known for its clear syntax, readability, and extensive standard library, making it the premier language for web development, data science, and AI/ML ecosystems. Compared to Java, Python enables drastically faster rapid prototyping and iterative development cycles due to its concise syntax and dynamic typing. It boasts a universally superior ecosystem for artificial intelligence and machine learning integrations.

Arelle

Users can interact with Arelle via an Interactive GUI.

Arelle processes HTML files by validating both the structure of the document and the XBRL data embedded inside it. When an iXBRL file is loaded into Arelle, the platform performs several steps:

It verifies contexts (entities, periods) and numeric units, ensuring they match standard taxonomies like US GAAP.

It runs checks specific to the selected filing manual (EFM for the US SEC) to catch missing values or incorrect data types.

It enforces calculation linkbases (checking totals, subtotals, and dimensional aggregations), while flagging inconsistencies or mathematical warnings.

Arelle is the premier tool relied upon by technology vendors and regulators for iXBRL data quality assurance.

Data Management & Security

SQLAlchemy

SQLAlchemy is an advanced Object-Relational Mapper (ORM) and SQL toolkit for Python, providing database administrators and developers with the full power and flexibility of SQL

inside an object-oriented design patterns format. Compared to Django ORM, SQLAlchemy is completely decoupled from any overarching web framework, offering far more granular control over complex database queries, multi-table joins, and intricate optimization patterns without requiring hacky workaround.

Pydantic

Pydantic is a powerful data validation and settings management library for Python that enforces type hints at runtime, providing user-friendly, structured error strings when validation fails. Compared to traditional standard JSON Schema validators or custom validation scripts, Pydantic is significantly faster (compiled via Rust in V2) and seamlessly coerces input data into strongly-typed Python objects, rendering it highly dependable for robust API request validation.

JWT (JSON Web Tokens)

JWT is an open industry standard (RFC 7519) used to securely transmit information between parties as a compact, self-contained JSON object, heavily used for stateless user authentication and authorization headers. Compared to Session-based authentication, JWTs completely eliminate the requirement to maintain a centralized session store or database on the server side. This architecture allows backend services to scale horizontally across servers instantly by validating cryptographic signatures locally.

PostgreSQL

PostgreSQL is a highly robust, open-source object-relational database management system (RDBMS) renowned for its extreme reliability, extensive feature set, and strict standards compliance. Compared to MySQL, PostgreSQL handles complex analytical queries, concurrent writes, and advanced data structures like JSONB arrays dramatically better. It functions flawlessly as a powerful hybrid relational-document database without sacrificing transactional ACID guarantees.

AI & Orchestration

LangGraph & LangChain

LangChain provides an extensive framework for building LLM-powered applications, while LangGraph extends it to build stateful, multi-actor, and highly cyclical agentic workflows. Compared to simple linear LLM chains, this combined stack enables developers to build complex, non-linear agent architectures capable of error-correction, looping reasoning paths, and maintaining state over massive, long-running processing pipelines.

ChatOllama

ChatOllama (via Ollama) is an integration interface that allows developers to host, serve, and run large language models locally on individual workstations or private infrastructure. Compared to cloud-hosted APIs like OpenAI GPT-4, ChatOllama ensures complete data

privacy and zero API usage costs by pulling, managing, and running open-source models (like Llama or Mistral) entirely within a secure, offline local machine context.

Claude

Claude is a family of cutting-edge foundational large language models engineered by Anthropic, meticulously designed for advanced reasoning, coding, and deeply contextual analysis tasks. Compared to GPT-4 in development applications, Claude models (particularly Claude Sonnet) display exceptionally superior code generation capability, more nuanced logical reasoning, and a much cleaner, less verbose technical writing style.

Gemini

Google Gemini is Google's multimodal AI family and assistant that understands text, images, code, audio, and video. It powers chat, Search summaries, Workspace, and developer tools for reasoning, writing, coding, and data analysis. Its competitive advantage is deep integration across Google Search, Android, Workspace, and Cloud, plus long-context reasoning and real-time, web-grounded information at global scale.

Meta AI (Llama)

Meta AI represents the suite of highly sophisticated, state-of-the-art open-weights large language models (Llama series) distributed openly to the global research and enterprise developer community. Compared to proprietary closed alternatives like Google Gemini or OpenAI, Meta's open weights empower organizations to fine-tune models on proprietary enterprise datasets and deploy them self-hosted on local infrastructure, ensuring total architectural independence.

Powershell

Windows PowerShell is Microsoft's native command-line shell and scripting language designed for task automation and system management. For Windows desktop applications, native scripts outperform Docker because they run directly on the host OS with zero virtualization overhead. They seamlessly interact with Windows APIs, the registry, and GUI elements, whereas running desktop apps through Docker adds unnecessary Linux/VM complexity.

Section 15: Discussion

While this prototype demonstrates functional capability for generating Form 10-Q and Form 10-K filings, commercialization requires careful consideration of structural and regulatory constraints. Although all public companies must file these forms, the underlying systems and presentation requirements vary significantly across organizations.

Key Limitations to Commercialization

The following issues limit the direct reusability of the current implementation as a standardized commercial product:

1. Design Deliberations: The prototype employs a frontend user interface (UI) for visualizing the financial statements and editing the MD&A and Notes narratives with internal AI assistance. The backend also has the function to merge all component docx files into a single Form 10-K or Form 10-Q report. What are some alternative designs?

Architecture Alternatives

The frontend is not strictly necessary: the backend can generate financial statements, write them directly to .docx, and merge all component files into a final Form 10-K or 10-Q report.

Narrative sections like MD&A and Notes could be edited directly in a standard word processor, with or without external AI assistance.

Conversely, if users want AI-assisted editing across other report sections, the frontend could be expanded into targeted mini-applications. Each mini-app would pair a dedicated frontend and backend to handle specific 10-K or 10-Q items (e.g., Legal Proceedings or Cybersecurity) whenever standard templates require customization.

Decision Options

- Backend-Only: Eliminate the custom frontend. Generate statements programmatically and edit narrative .docx files directly using standard word processors.
- Status Quo: Retain the existing combined frontend and backend to support current financial statement visualization and MD&A and Notes editing workflows.
- Modular Expansion: Build out focused, item-specific mini-applications with tailored frontend and backend AI capabilities.
- All of the Above: Implement any option at the customer's discretion.

2. Database Interface Variability: The prototype relies on a simulated database that approximates real financial data. In practice, accounting databases differ substantially in schema design, chart of accounts, data models, and governance practices. As a result, the database

interface developed for this demonstration is unlikely to transfer to another client without significant re-engineering.

3. Financial Statement Presentation Diversity: Although the Securities and Exchange Commission (SEC) prescribes a standard item structure for Form 10-K filings, companies exercise considerable discretion in formatting. Balance sheets, income statements, and cash flow statements exhibit wide variation in line-item grouping, terminology, level of detail, and footnote integration. This diversity constrains the reusability of the front-end presentation layer.

4. Backend Processing Constraints: Given the wide variation in filing formats and the limited reusability of both the data access layer and the user interface, the intermediate backend logic—which transforms raw accounting records into filing-ready structures—is also unlikely to be portable across implementations without modification.

5. LLM-Assisted Development Considerations: The application was created with assistance from large language models (LLMs). For filings of regulatory significance, stakeholders may question the traceability, validation, and auditability of LLM-generated code. While comprehensive testing can mitigate risk, acceptance of this methodology may require additional governance and documentation compared to traditional software engineering.

6. Runtime Generation of MD&A and Notes: Two critical components of Form 10-K, the MD&A and Notes, are currently drafted at runtime using an LLM (ChatOllama) to populate narratives for subsequent human editing. Unlike the development-time use of Claude, which does not execute during operation, ChatOllama operates within the live application. This introduces questions regarding accuracy, consistency, and regulatory defensibility. The requirement for human review provides a mitigating control, but does not eliminate reliance on generative output for material disclosures.

7. iXBRL Notes Issue: Beyond the reliability of LLM-generated narrative in Form 10-K notes, inline XBRL (iXBRL) tagging presents a major structural challenge. Because the disclosure requirements and number of notes vary by company, predicting which iXBRL tags will be required is highly difficult. For this simplified application, the backend codebase is pre-configured with a suite of the 8 most common financial notes, but needs to be extended for any particular real company. Users select the specific notes required for a given filing cycle (e.g., selecting 20 notes in Year 1 and 17 notes in Year 2).

Strategic Opportunity

Despite these limitations, the project highlights a compelling use case for LLM-accelerated development. Because each company maintains a unique accounting database and adopts a customized filing format, a one-size-fits-all product is impractical. Tailored implementations

are necessary, and LLMs offer a distinct speed advantage for creating such made-to-order solutions.

Two viable approaches emerge:

1. Iterative Adaptation: Prompt the LLM to modify the existing codebase to accommodate a specific client's data model, filing presentation requirements, and initial notes baseline.
2. Greenfield Generation: Direct the LLM to generate a new application, or multiple mini-applications, from first principles for each client, incorporating lessons learned from prior implementations.

The demonstration confirms technical feasibility but also underscores why direct commercialization as a standardized product is not advisable. The value lies not in reuse, but in rapid customization—an area where LLM-assisted development provides meaningful competitive advantage, provided appropriate human oversight is maintained for regulatory content, particularly in the case of the MD&A and Notes.

Conclusion: Key Takeaways for LLM-Assisted Development

This application demonstrates the capacity of Large Language Models (LLMs) to significantly accelerate development cycles by generating both functional code and realistic sample data. However, maximizing the efficiency of these tools requires a nuanced understanding of prompting dynamics and iterative strategies.

Notably, this project relied exclusively on the free tiers of the selected LLMs, which delivered very satisfactory performance. However, Claude Sonnet 5 occupies a category of its own; while its free version proved highly skilled, it was constrained by a strict rate limit of one prompt every four or five hours. Upgrading to a premium tier would have significantly mitigated this bottleneck and improved development turnaround times. Therefore, it is impossible to estimate the total number of hours to create this application.

LLM Summary

- **Meta AI:** Successfully generated the Python code required to build a standard accounting spreadsheet with simulated data, which was imported into the application's database. Used the “Thinking” level in the Meta prompt interface. The “Contemplating” level did not work well for coding. Meta AI also wrote portions of this document.
- **Gemini:** Drafted the vast majority of this document with high accuracy, and created significant improvements to the database schema. It also helped to improve the instructions prompting the Meta AI task, and suggested refactoring the Notes python code to overcome the shortcomings of ChatOllama models. Used 3.6 Flash model with Extended Thinking option.
- **ChatOllama:** Effectively authored the Management's Discussion & Analysis (MD&A) and Notes to Financial Statements for both the Form 10-Q and Form 10-K reports. Recall ollama models are downloaded to a local computer, so that the application can run without any internet connection. Choosing an ollama model depends on the amount of VRAM/RAM available. This application has been tested with 16GB RAM, and 8GB GPU (VRAM). Based on the struggle Claude had with MD&A and Notes processing, it would be worthwhile to experiment with a larger GPU and upgraded model to compare narrative results. Used ollama qwen2.5:7b model with 8GB GPU.
- **Claude:** Delivered high-quality Python and React code to power the core application, as well as a file transformer to add iXBRL tags to HTML. It also significantly improved the

instructions for prompting the Meta AI task. Used Sonnet 5 model with Medium level thinking option.

Best Practices for Effective Prompts

- **Be Specific:** Because LLMs strictly follow instructions, prompts must be highly detailed and exact to achieve the desired output. If the LLM seems to misinterpret the prompt, ask a different LLM to check the prompt for accuracy. In the current project, Claude corrected serious flaws in the instructions given to Meta AI to generate the dataset.
- **Manage Complexity:** Avoid overloading a single prompt with excessive detail. Too much dense complexity can increase processing ("thinking") time or cause the model to fail to provide a response entirely.
- **Leverage Specification Files:** Instead of typing exceptionally long prompts, use platforms or environments that support attaching reference and specification files directly to the project.
- **Assume no Memory:** Even though the LLM generates most of the files for a project, do not assume they are saved in the LLM datastore. When making code updates or fixes, upload the existing files in case the LLM has "forgotten" them.

Managing the Iteration and Refinement Process

- **Anticipate Iteration:** Initial outputs may not perfectly align with project requirements, which can cause development frustration. Overcome this by continually refining and iterating on the prompts until the desired outcome is achieved.
- **Avoid Manual Debugging:** When the LLM generates code that fails or produces incorrect output, avoid fixing it manually. Instead, write a prompt detailing the exact issue and attach a file containing any error messages. Note that Claude Sonnet has proven particularly effective at **self-correction** in this application.
- **Speed vs. Quality: The LLM Illusion**

Don't let rapid-fire delivery fool you; a fast response isn't always a finished one.

- **The Mirage:** The AI spits out a complete answer in minutes.
- **Hidden Cost:** You spend the next several hours (or days) prompt-engineering fixes for hidden mistakes, glaring omissions, and unwanted "features."

The Takeaway: If a "fast" output requires dozens of iterations to actually become useful, it didn't save you time - it just rescheduled the work. Avoid this situation by redesigning the prompts, or restarting the LLM conversation.

- **Handle Minor Tweaks Carefully:** When an output is nearly complete and requires only minor adjustments, prompt the LLM to make those specific tweaks. Remember to update your original specification files simultaneously so they remain accurate for future use.
- **Know When to Restart:** Accumulating too many small tweaks can lead to overly convoluted responses in which the LLM attempts to patch together multiple updates. If the structure becomes messy, it is sometimes more effective to update the core specification files and restart the project fresh from scratch.

Case Study: Second Opinion

If an LLM gets stuck in a loop of applying band-aid fixes to failing code, bring in a second model for a fresh perspective.

When automating iXBRL tagging for 10-K Notes, Claude struggled to handle weak narrative outputs from ChatOllama, constantly offering superficial workarounds that failed. Seeking a second opinion from Gemini broke the cycle. Gemini identified the root structural issues and recommended three key fixes:

- Loop Refactoring: Process each note individually within a loop rather than all notes at once, to avoid issues with (1) token memory window and (2) maximum response length.
- Model Upgrade: Upgrade the underlying Ollama model for better narrative quality.
- Parameter Tuning: Pass enhanced arguments into the ChatOllama function.

Case Study: Strategic Guidance vs. Blind Execution

After making changes to an mda agent, a developer tasked Claude with refactoring a related notes agent. When Claude began updating the Python code, the developer interrupted and instructed Claude to log internal execution data first. This exchange highlights four key principles of developer-AI interaction:

1. LLMs lack proactive foresight: Focused solely on refactoring, the model didn't independently consider logging diagnostic data first.
2. Unchecked execution leads to wasted work: Without human intervention directing the AI to inspect logs, the LLM would have blindly rewritten the codebase.

3. Context drives reasoning: Once Claude analyzed the logged data, it concluded that refactoring was completely unnecessary.
4. The developer must **think** carefully before prompting the LLM to do any major code modifications.

Even though LLMs can effectively process new information to adapt their approach, they rely on human developers to provide the right context and ask the necessary questions.

The Developer's Ultimate Role

While LLMs dramatically accelerate code implementation, they cannot replace the core responsibilities of a software engineer. Developers must still write clear design specifications, direct system architecture, and remain accountable for identifying and correcting AI-generated mistakes.

LLM Impact

Despite headlines suggesting AI is replacing software engineers, companies cutting headcount are missing a better alternative: shortening the workweek. For instance, if AI yields a 20% boost in coding productivity, a company could reduce working hours by 20% while maintaining the exact same output. Rather than cutting salaries by 20% to match those reduced hours, employers could offer a 10% hourly raise. This creates a win-win scenario: the company captures a 10% overall gain, and employees enjoy significantly more leisure time with only a minor impact on total income—ultimately boosting morale and retention.

AI References

<https://fortune.com/2026/08/10/jersey-pump-principle-ai-bet-could-stall-history/>

<https://finance.yahoo.com/technology/ai/articles/layoffs-tied-ai-hurt-worker-130409156.html>

<https://siepr.stanford.edu/publications/policy-brief/what-really-happening-jobs-separating-ai-hype-reality>

<https://www.ciodive.com/news/leaders-await-revenue-boost-ai-2030/810173/>